



CS 584 Natural Language Processing

Lecture 5: Seq2seq Models and Machine Translation

Yue Ning
Department of Computer Science
Stevens Institute of Technology

Learning Objectives

- ❖ Understand Seq2seq Models
- ❖ Be able to apply Seq2seq models
- ❖ Understand attention mechanism

Overview

- ❑ Introduce a new task: Machine Translation

↓ is a major use-case of

- ❑ Introduce a new neural architecture: Seq2seq models

↓ is improved by

- ❑ Introduce a new neural technique: Attention

Machine Translation

- **Definition** of Machine Translation (MT)
- Example:
x: have a good day (English)
↓
y: bonne journée (French)

Machine Translation

- Human translation vs. machine translation
- Why is it hard for computer programs?

History of Machine Translation

- 1950s: Early Rule-based Machine Translation (**RBMT**)
What is the motivation?
- 1990s-2010s: Statistical Machine Translation (**SMT**)
- 2014- : Neural Machine Translation (**NMT**)

Statistical Machine Translation

- Core idea: learn a probabilistic model from data
- French **f (source)** → English **e (target)**

Mathematically:

$$P(e|f) = \frac{P(e) P(f|e)}{P(f)}$$

Translation $T(f) = \hat{e} = \operatorname{argmax}_e P(e|f) = \operatorname{argmax}_e P(e) P(f|e)$

Statistical Machine Translation

Components

$$T(f) = \hat{e} = \text{argmax}_e \underbrace{P(e)}_{\text{Language Model}} \underbrace{P(f|e)}_{\text{Translation Model}}$$

- Language Model
- Translation Model
- argmax

The language model $P(e)$

$$T(f) = \hat{e} = \operatorname{argmax}_e \underline{P(e)P(f|e)}$$

Estimation of how probable a sentence is.

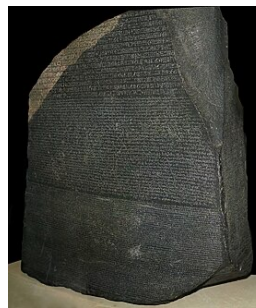
- Refer to the previous slides

The translation model $P(f|e)$

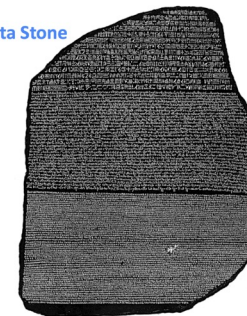
$$T(f) = \hat{e} = \operatorname{argmax}_e P(e) \underline{P(f|e)}$$

Estimation of the lexical correspondence between languages.

- How to learn translation model $P(f|e)$?
 - First, need large amount of parallel data (e.g. pairs of human-translated French/English sentences)



The Rosetta Stone



Ancient Egyptian

Demotic

Ancient Greek

The translation model $P(f|e)$

$$T(f) = \hat{e} = \operatorname{argmax}_e P(e) \underline{P(f|e)}$$

Estimation of the lexical correspondence between languages.

- How to learn translation model $P(f|e)$ from the parallel corpus?

- Break it down further: we actually want to consider

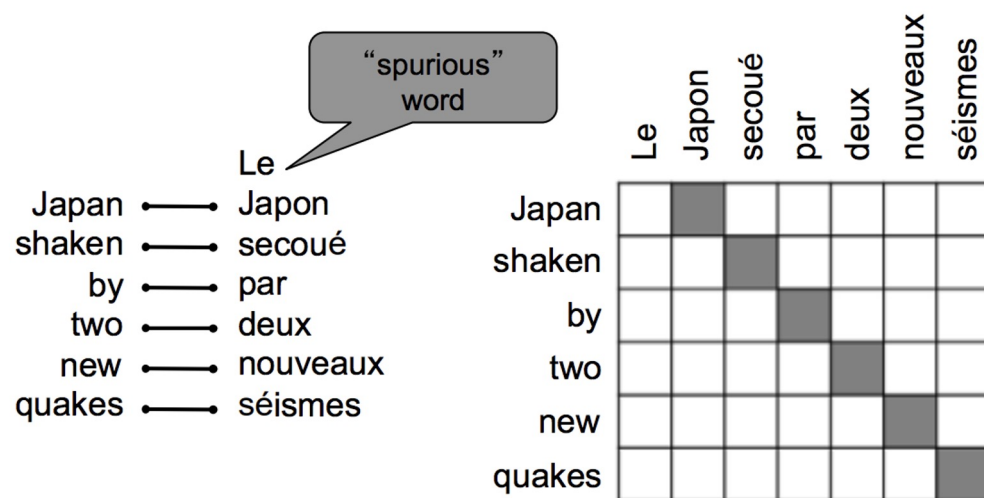
$$P(f, a|e)$$

where a is the **alignment**, i.e. word-level correspondence between French sentence x and English sentence y

What is alignment?

Alignment is the correspondence between particular words in the translated sentence pair.

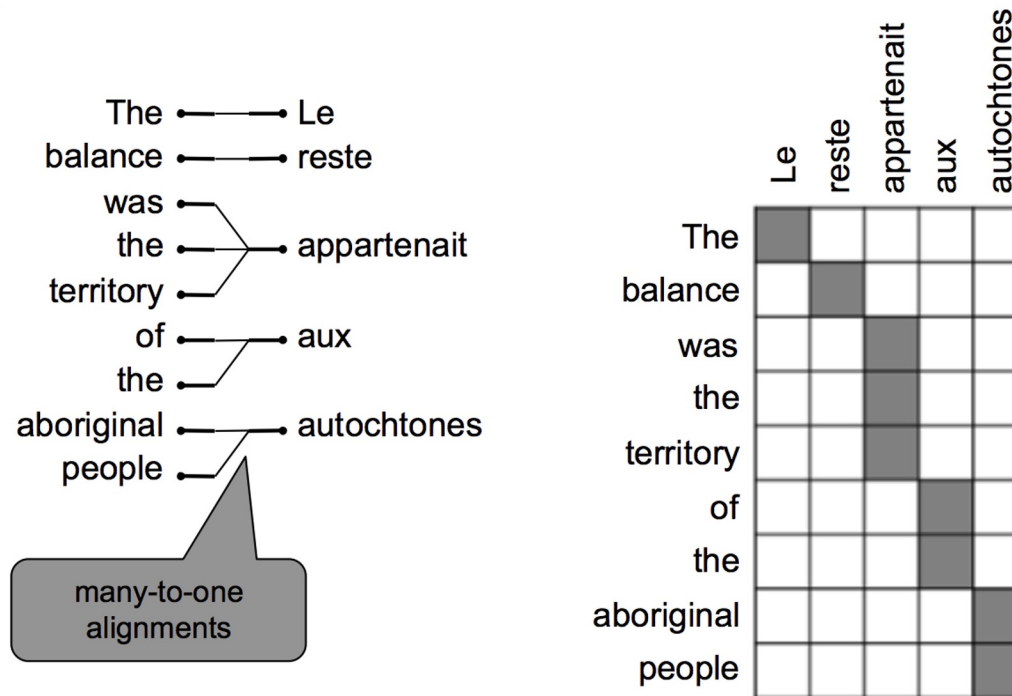
- Note: Some words have **no counterpart**



Examples from: "The Mathematics of Statistical Machine Translation: Parameter Estimation", Brown et al, 1993.
<http://www.aclweb.org/anthology/I93-2003>

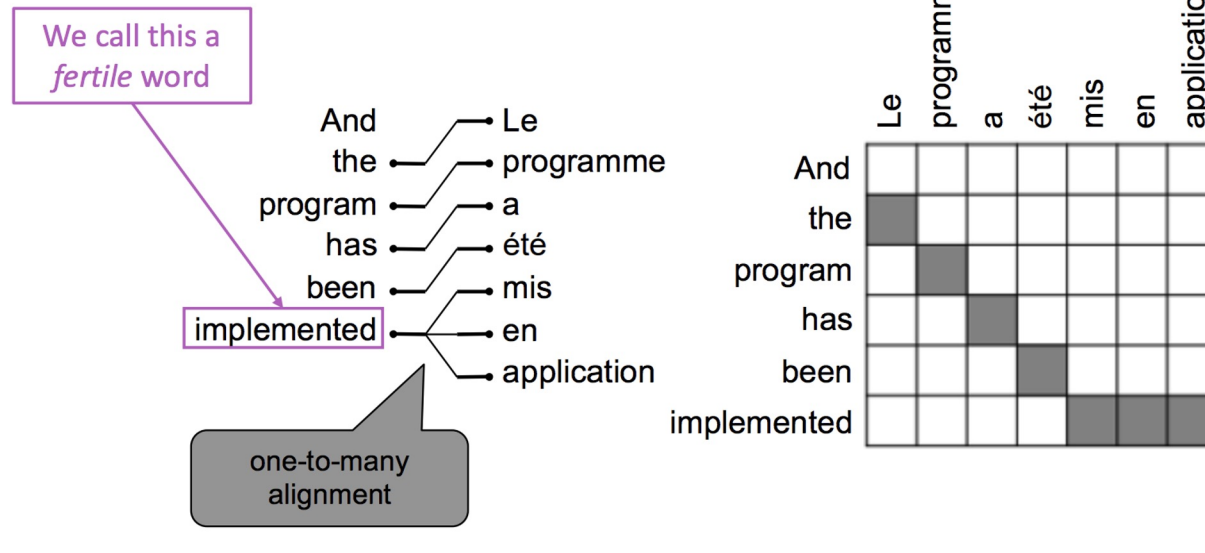
Alignment is complex

Alignment can be many-to-one



Alignment is complex

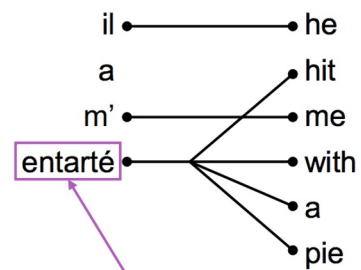
Alignment can be one-to-many



Alignment is complex

Some words are very fertile!

*correction: il m'a entarté



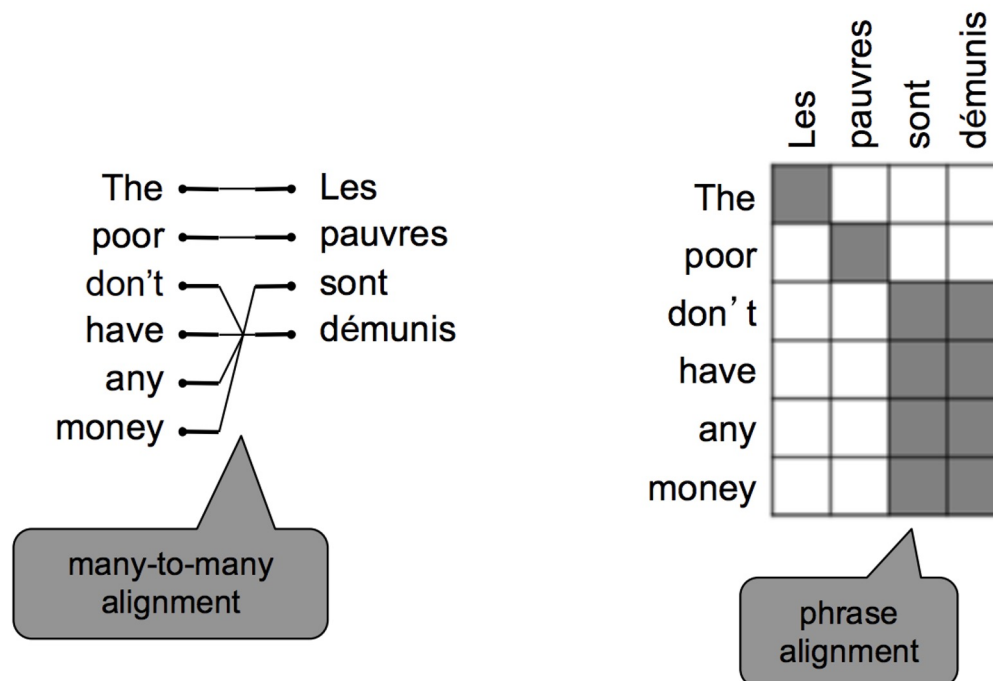
This word has no single-word equivalent in English

	he	hit	me	with	a	pie
il						
a						
m'						
entarté						



Alignment is complex

Alignment can be many-to-many (phrase-level)

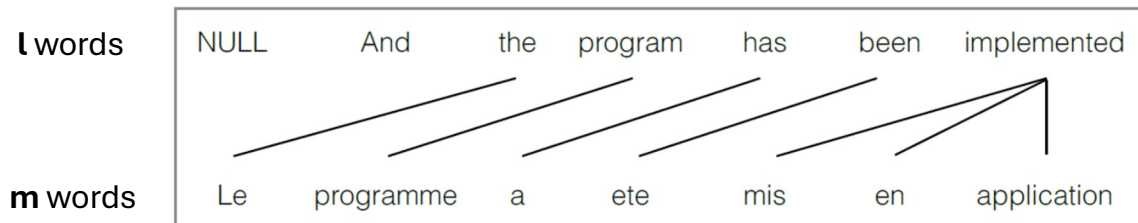


Learning alignment for SMT

- We learn $\overline{P(f, a|e)}$ as a combination of many factors, including:
 - Probability of particular words aligning (also depends on **positions** in a sentence)
 - Probability of particular words having particular fertility (**number** of corresponding words)
 - And others
- Next
 - A basic example of learning alignment

Word Alignment - Vector Representation

- Assumption: each French word **f** is aligned to exactly one English word **e**
 - Including NULL



- Alignment vector **a** = [2,3,4,5,6,6,6]
 - length of **a** = length of sentence **f**
 - $a_i = j$ if French position i is aligned to English position j
- How many possible alignments in A?

$$(l + 1)^m$$

Formalizing the connection between word alignments & the translation model

$$\begin{aligned} & p(f_1, f_2, \dots, f_m \mid e_1, e_2, \dots, e_l, m) \\ &= \sum_{a \in A} p(f_1, \dots, f_m, a_1, \dots, a_m \mid e_1, \dots, e_l, m) \end{aligned}$$

- We define a conditional model
 - Projecting word translations
 - Through alignment links

IBM Model 1

- Input
 - an English sentence of length l
 - a length m
- For each French position i in $1..m$
 - Pick an English source index j
 - Choose a translation

Alignment
probabilities
are UNIFORM

$$q(j | i, l, m) = \frac{1}{l + 1}$$
$$\frac{t(f_i | e_{a_i})}{t(f_i | e_{a_i})}$$

Words are
translated
independently

$t(f|e)$

- $t(f|e)$
 - Word translation probability table
 - for all words in French & English vocab

f	e	$p(f e)$
le	the	0.42
la	the	0.4
programme	the	0.001
a	has	0.78
...	...	...

IBM Model 1

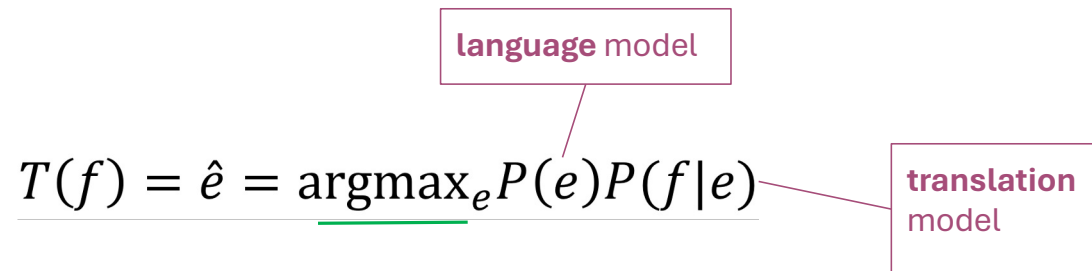
- Input
 - an English sentence of length l
 - a length m
- For each French position i in $1..m$
 - Pick an English source index j
 - Choose a translation

$$q(j | i, l, m) = \frac{1}{l + 1}$$
$$\frac{t(f_i | e_{a_i})}{t(f_i | e_{a_i})}$$

$$p(f_1 \dots f_m, a_1 \dots a_m | e_1 \dots e_l, m) = \prod_{i=1}^m q(a_i | i, l, m) t(f_i | e_{a_i})$$

- More: IBM Model 2
 - Remove assumption that q is uniform

Decoding for SMT

$$T(f) = \hat{e} = \text{argmax}_e P(e)P(f|e)$$


language model

translation model

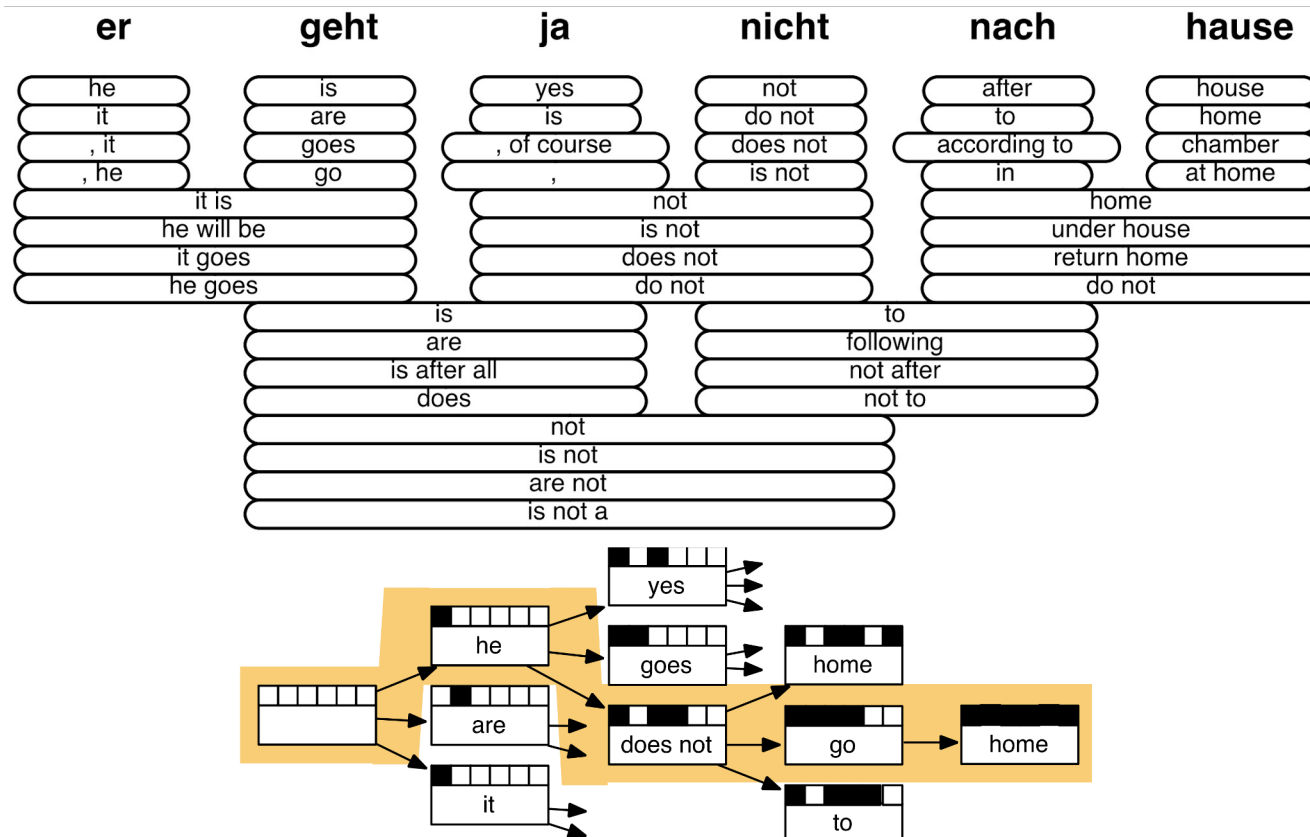
Responsible for the search in the space of possible translations.

- How to compute this **argmax**?
 - We could enumerate every possible e and calculate the probability? → Too expensive!
 - Answer: Use a **heuristic search algorithm** to **search for the best translation**, discarding hypotheses that are too low-probability
 - This process is called **decoding**

Decoding for SMT

er	geht	ja	nicht	nach	hause
he	is	yes	not	after	house
it	are	is	do not	to	home
, it	goes	, of course	does not	according to	chamber
, he	go	,	is not	in	at home
it is		not		home	
he will be		is not		under house	
it goes		does not		return home	
he goes		do not		do not	
	is		to		
	are		following		
	is after all		not after		
	does		not to		
	not				
	is not				
	are not				
	is not a				

Decoding for SMT



Source: "Statistical Machine Translation", Chapter 6, Koehn, 2009

Statistical Machine Translation

- Problems and Limitations of SMT
 - Extremely complex
 - Lots of feature engineering
 - Require compiling and maintaining resources
 - Lots of human effort to maintain

NMT: the first big success story of NLP Deep Learning

Neural Machine Translation went from a **fringe research attempt** in **2014** to the **leading standard method** in **2016**

- **2014**: First seq2seq paper published
- **2016**: Google Translate switches from SMT to NMT – and by 2018 everyone has



- **This is amazing!**
 - **SMT** systems, built by **hundreds** of engineers over many **years**, outperformed by NMT systems trained by a **small group** of engineers in a few **months**

Neural Machine Translation

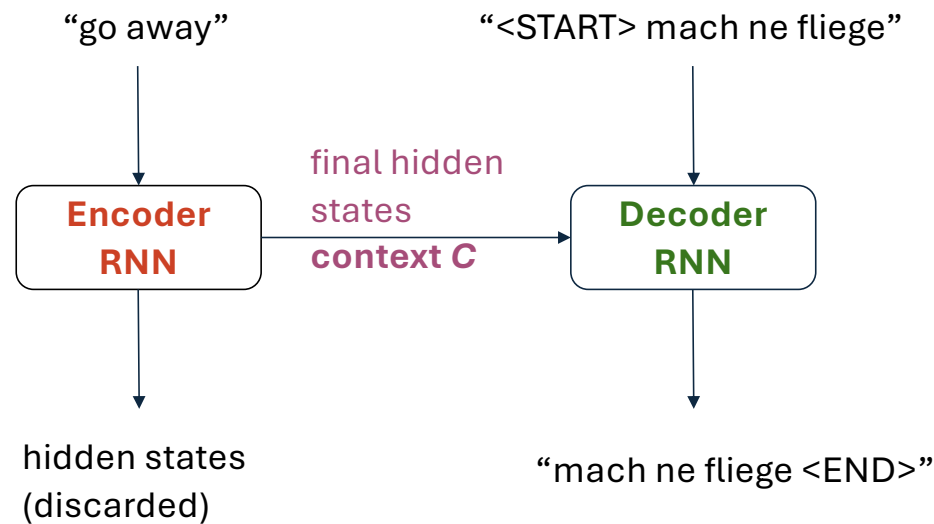
- A single **end-to-end** neural network
- A new architecture: **sequence-to-sequence (seq2seq)**
- Both the input and output are sequences.

Seq2Seq Basics

- [Sutskever et al. 2014](#) (English-French translation)
 - Side story: **who is** [Ilya Sutskever](#)?
- An end-to-end model made up of **two RNNs**
- Seq2Seq models are often referred to as "**encoder-decoder**" models

Seq2Seq

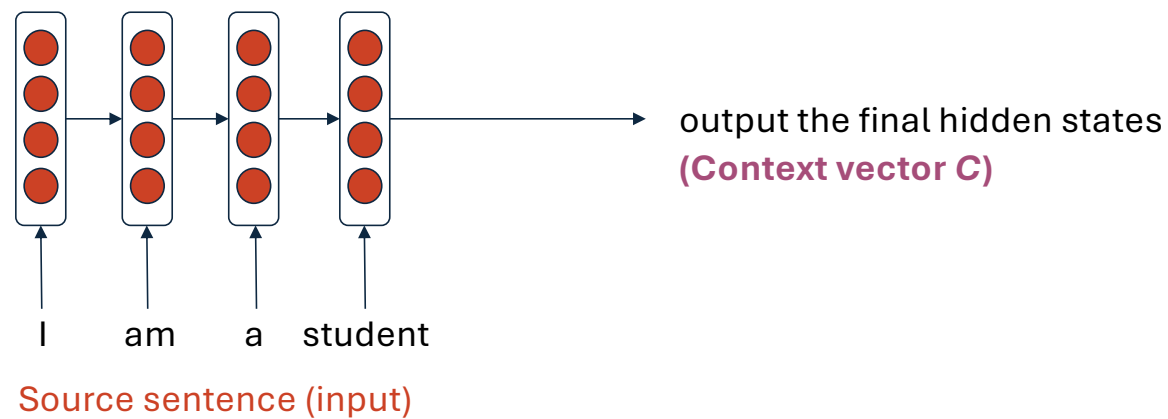
- 2 parts: **encoder** and **decoder**.



- The decoder is a **text generator**.
- Difference from the simple text generator: the **initial states** are determined by the encoder.

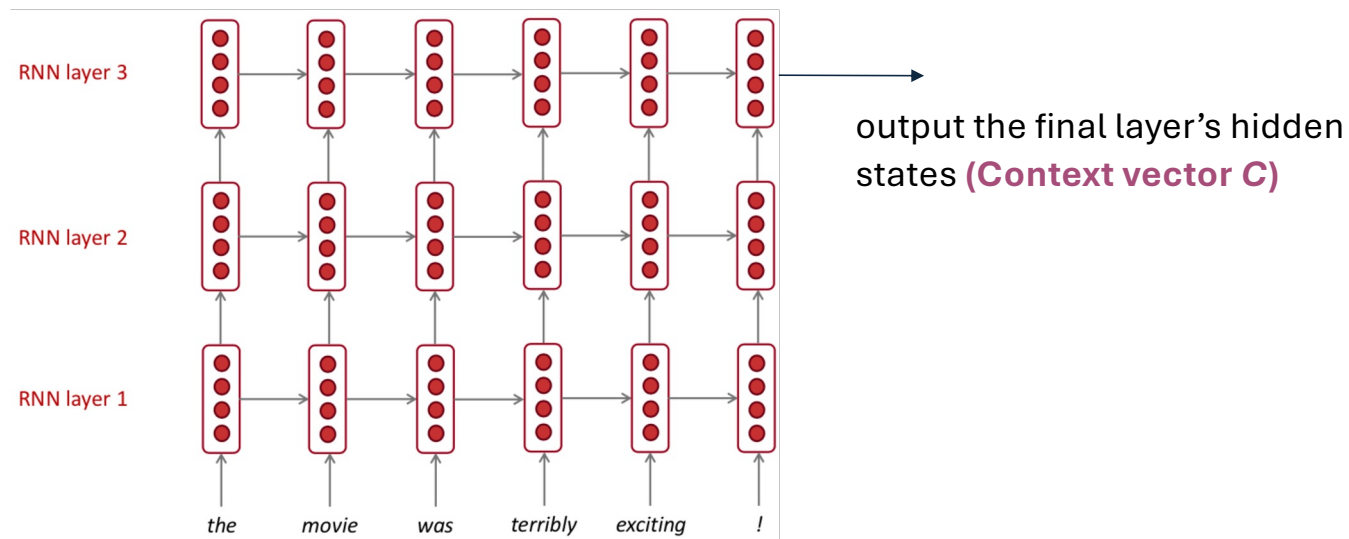
Seq2Seq - Encoder

- **Goal:**
 - read the input sequence and generate a **context vector C** for the sequence.
- How does it work?
 - using an RNN (e.g., LSTM or GRU)



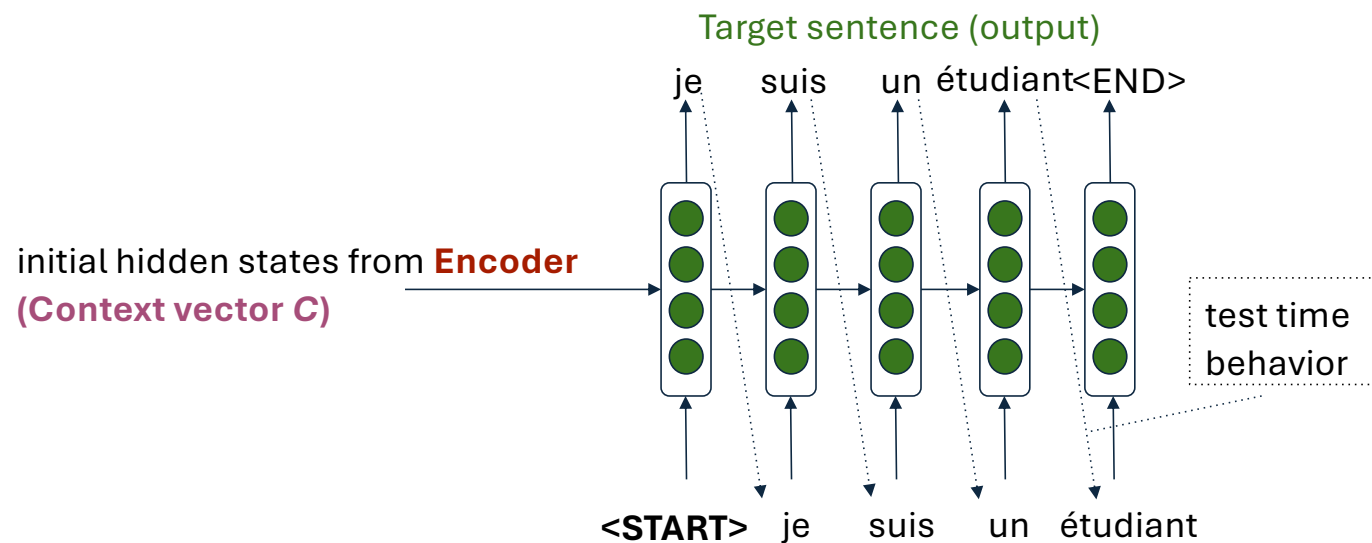
Seq2Seq - Encoder

- Problems/limitations
- Potential solution: the encoder will usually consist of **stacked RNNs**



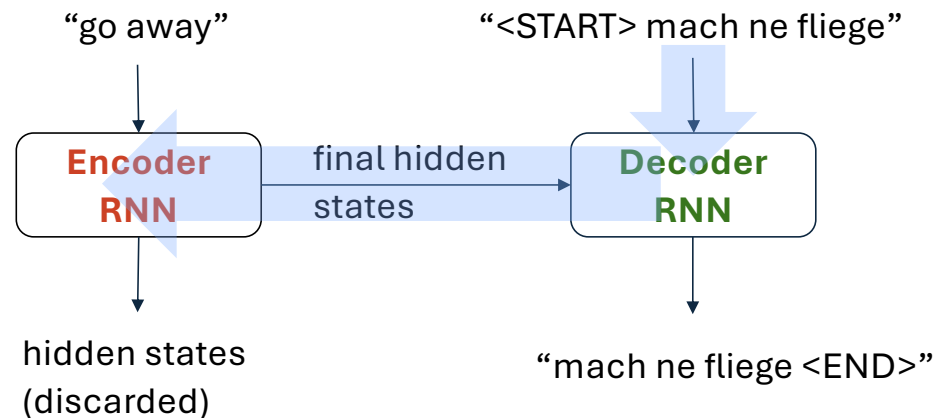
Seq2Seq - Decoder

- Goal: use the context vector to generate an output.
- How does it work?
 - Using another RNN

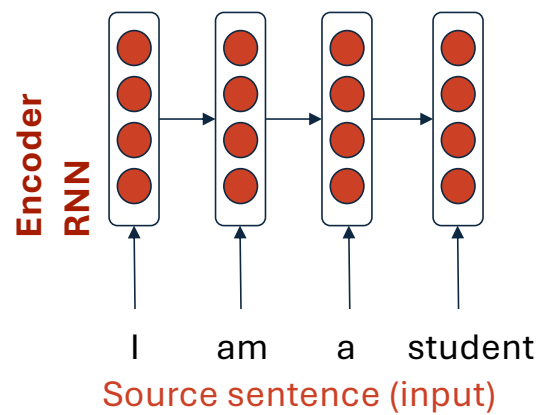


Seq2Seq - Training

- Loss: Cross-entropy
- Optimization: Backpropagation
- How it works: Both the **encoder** and **decoder** are trained at the same time, so that they both learn the same **context vector** representation.

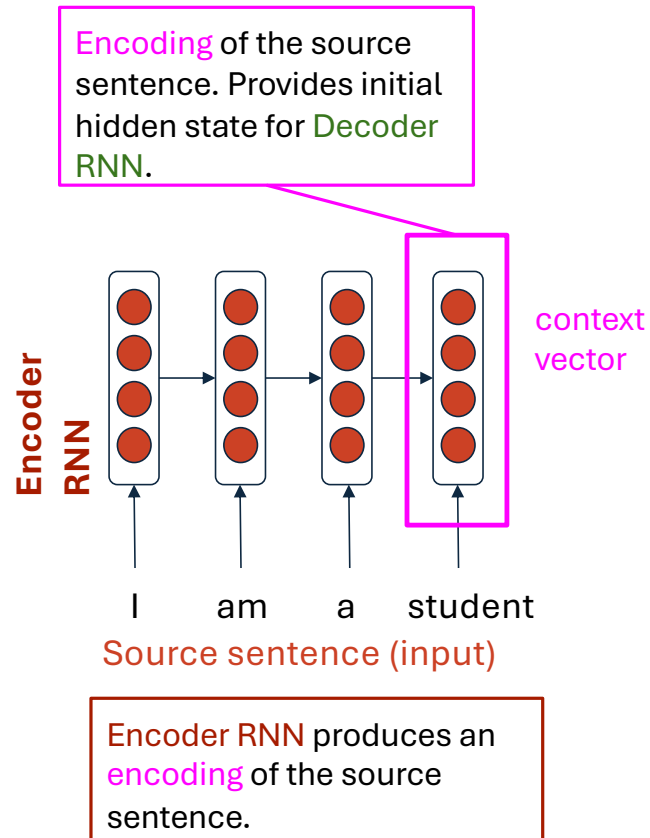


Neural Machine Translation - Seq2Seq

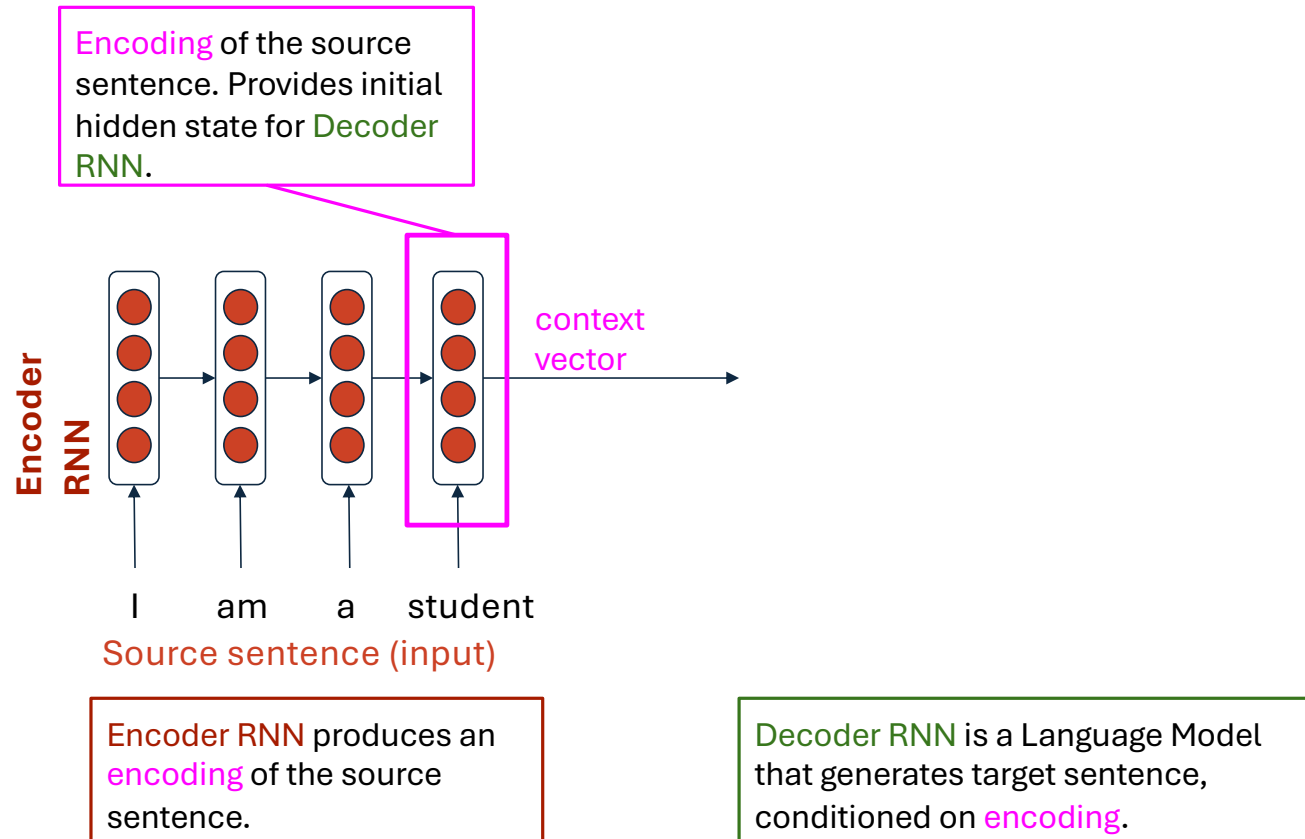


Encoder RNN produces an **encoding** of the source sentence.

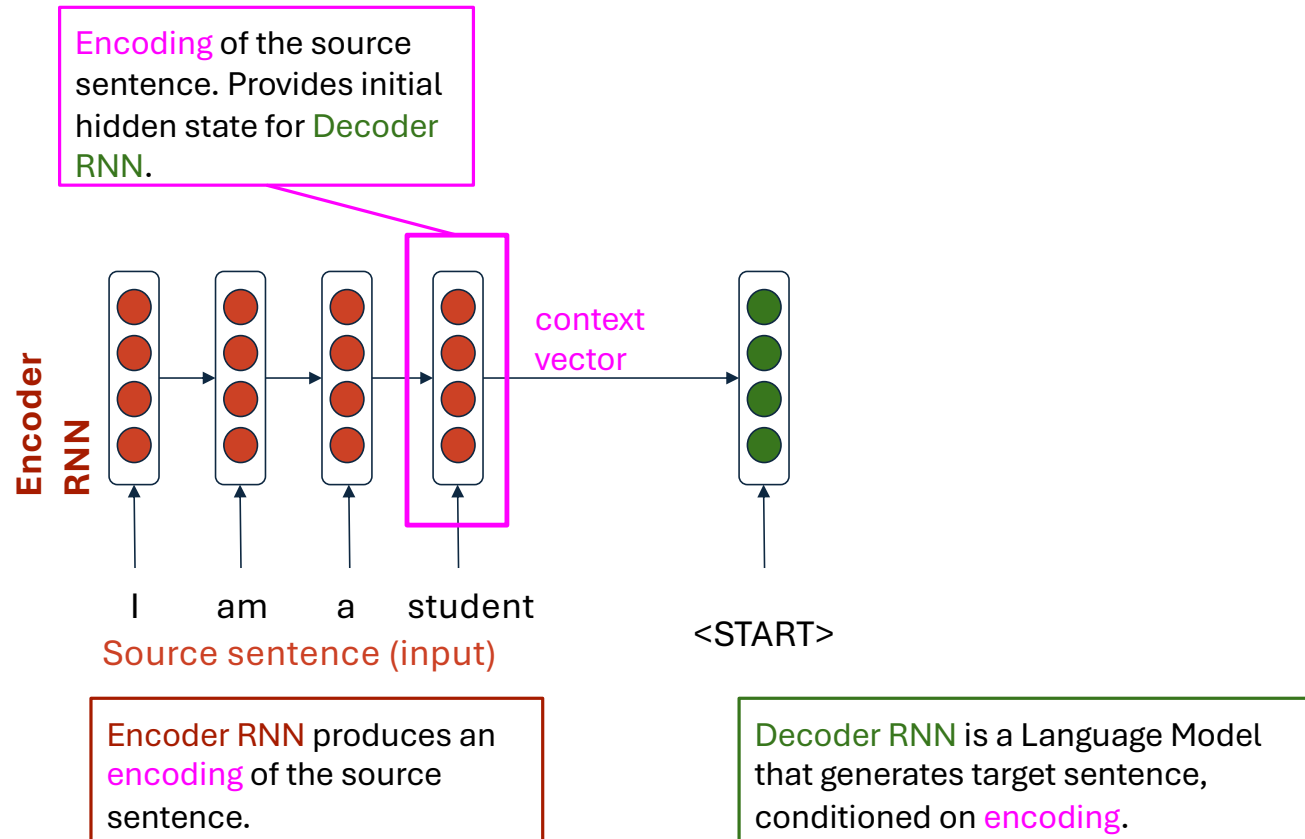
Neural Machine Translation - Seq2Seq



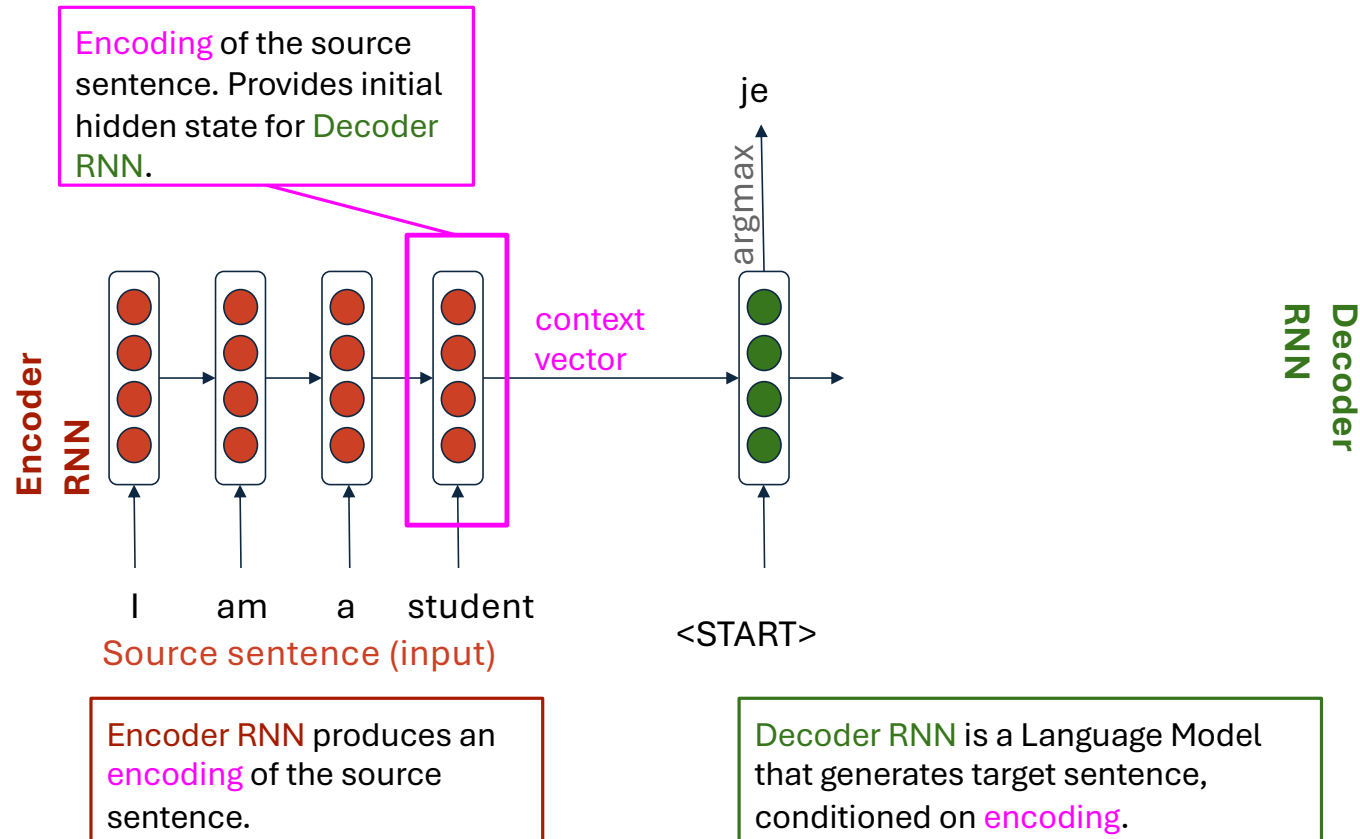
Neural Machine Translation - Seq2Seq



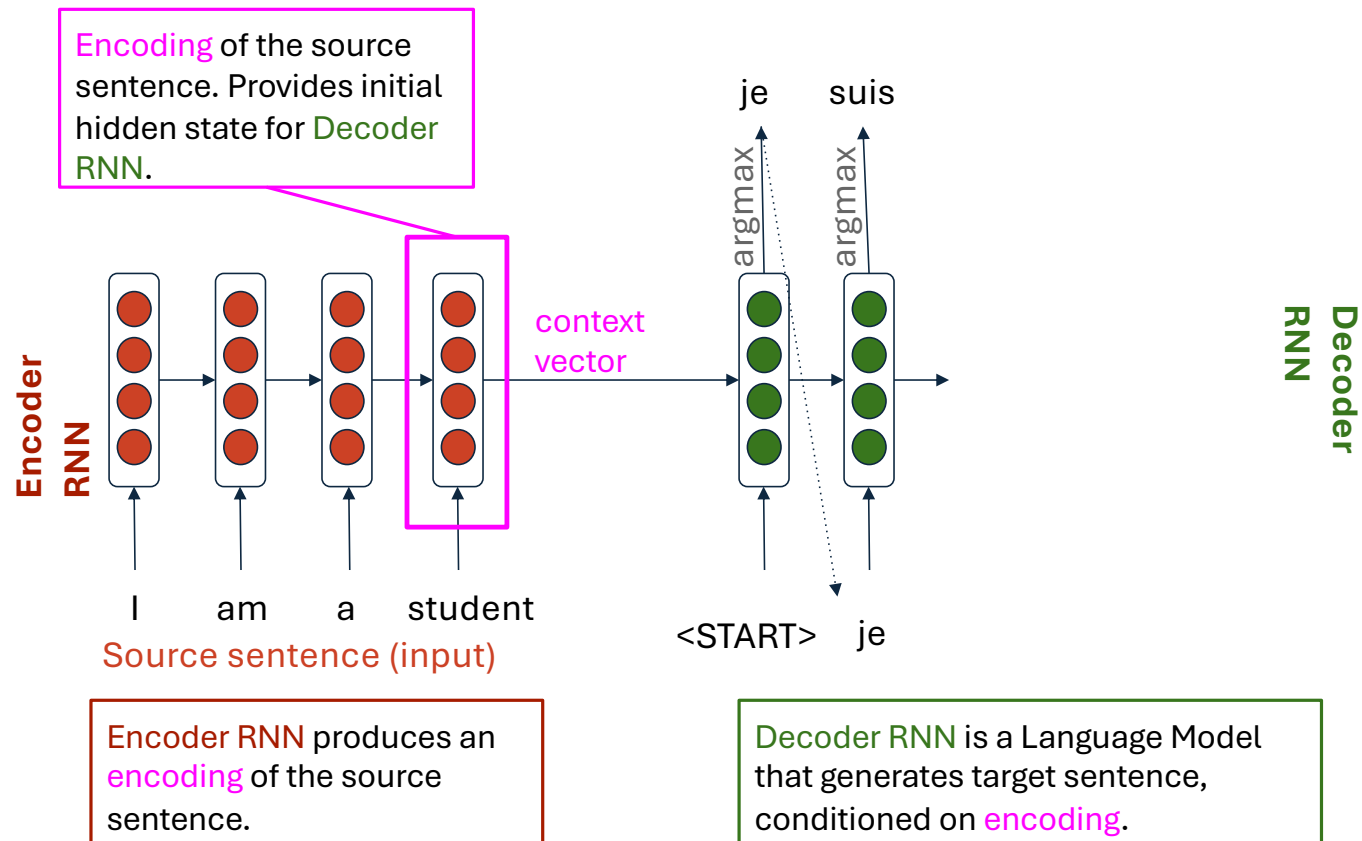
Neural Machine Translation - Seq2Seq



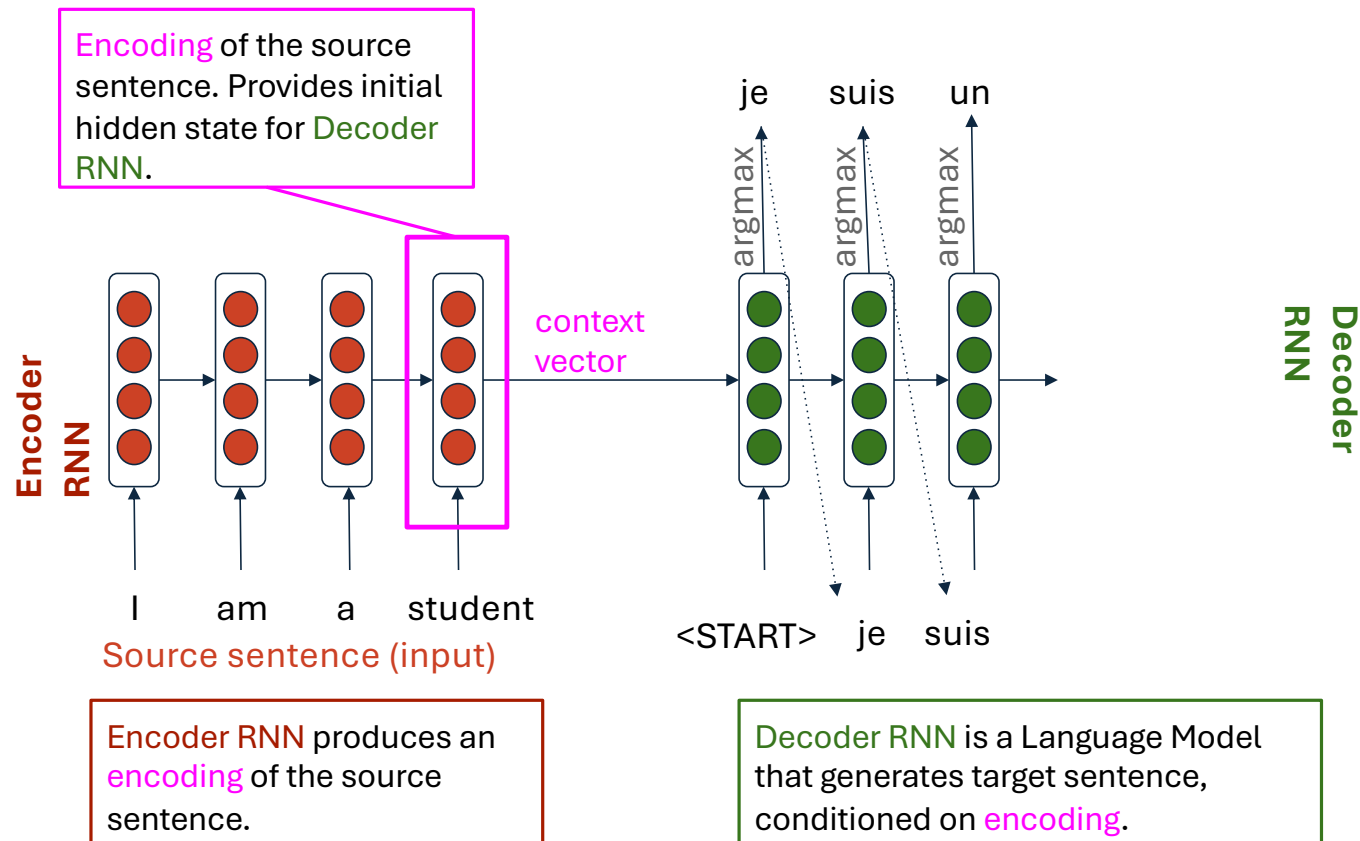
Neural Machine Translation - Seq2Seq



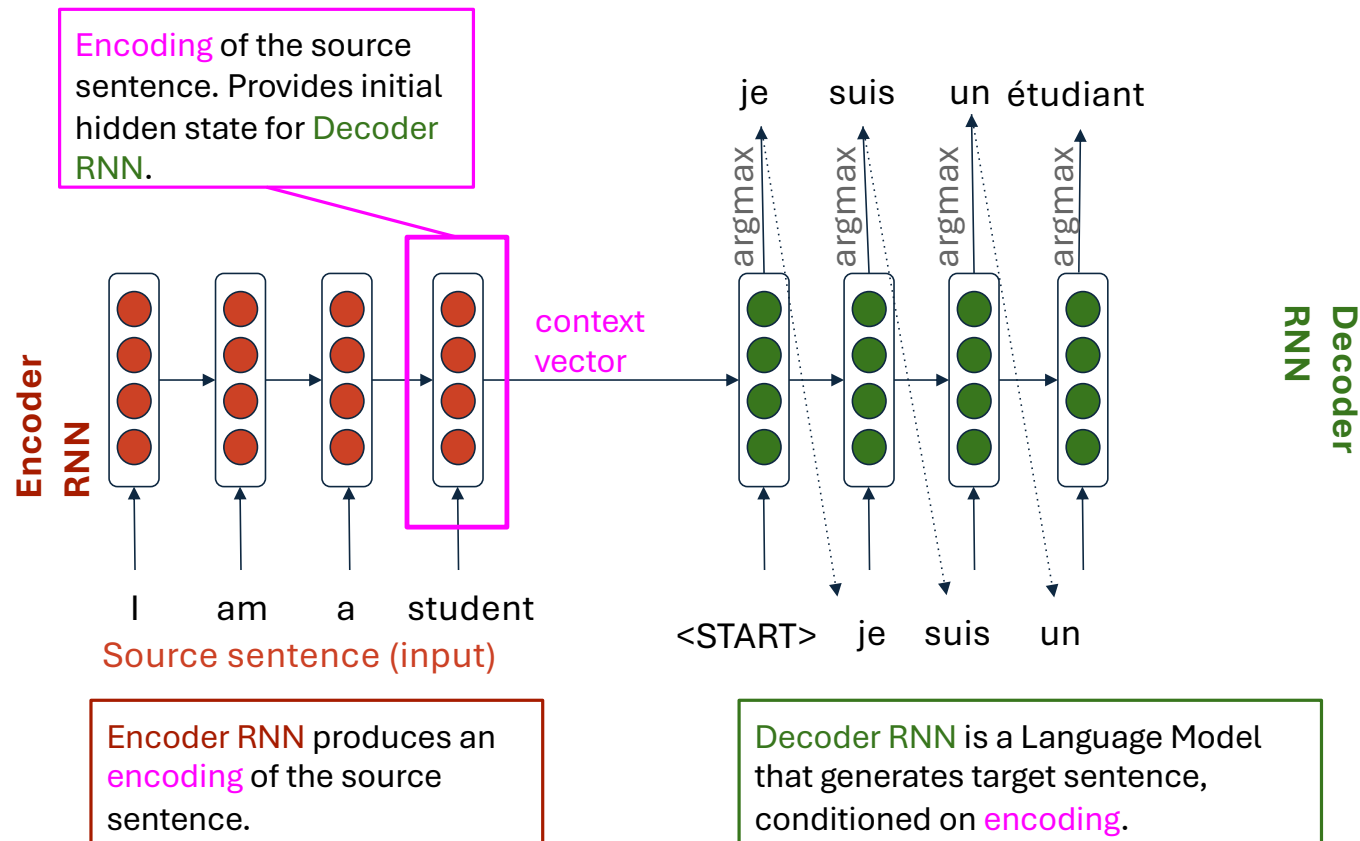
Neural Machine Translation - Seq2Seq



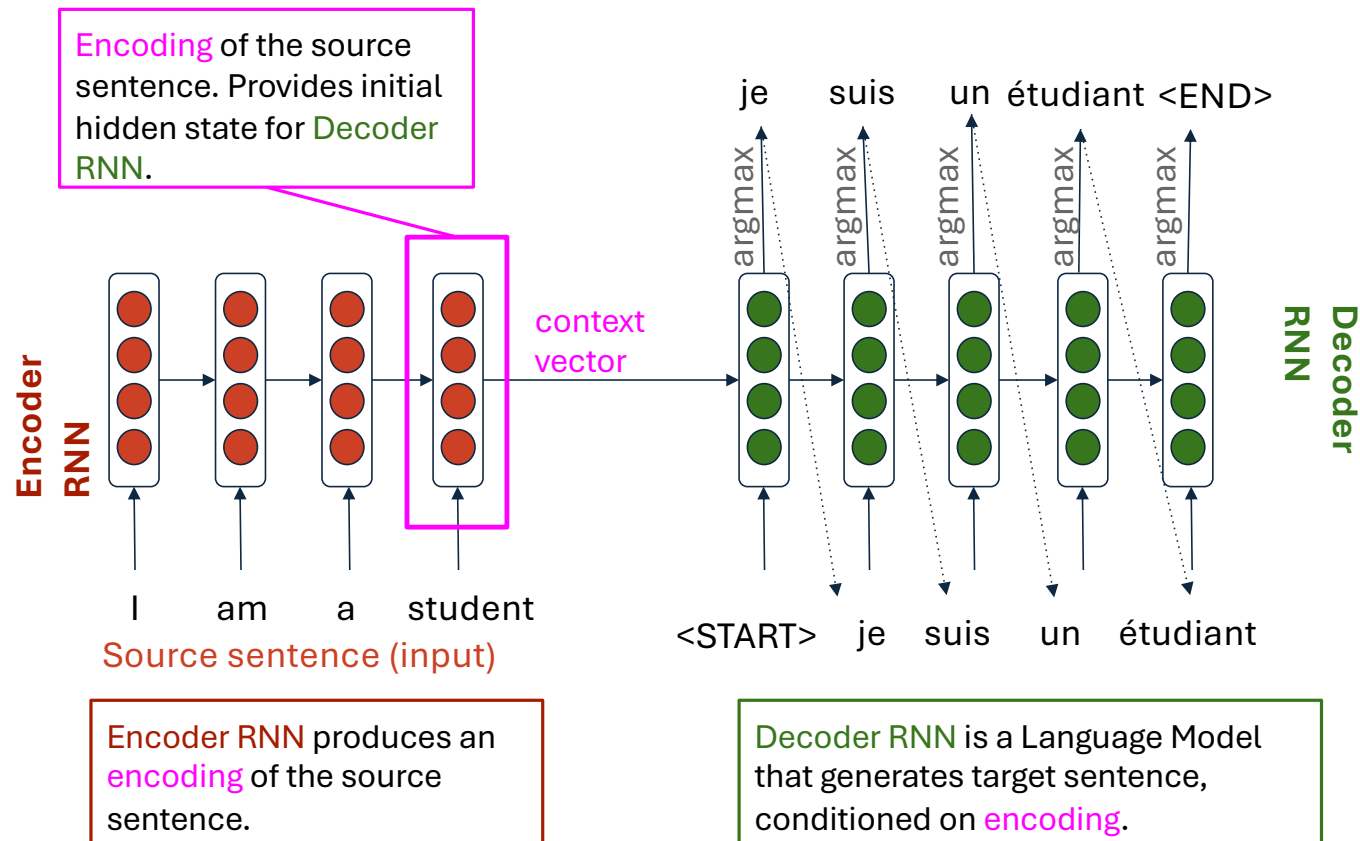
Neural Machine Translation - Seq2Seq



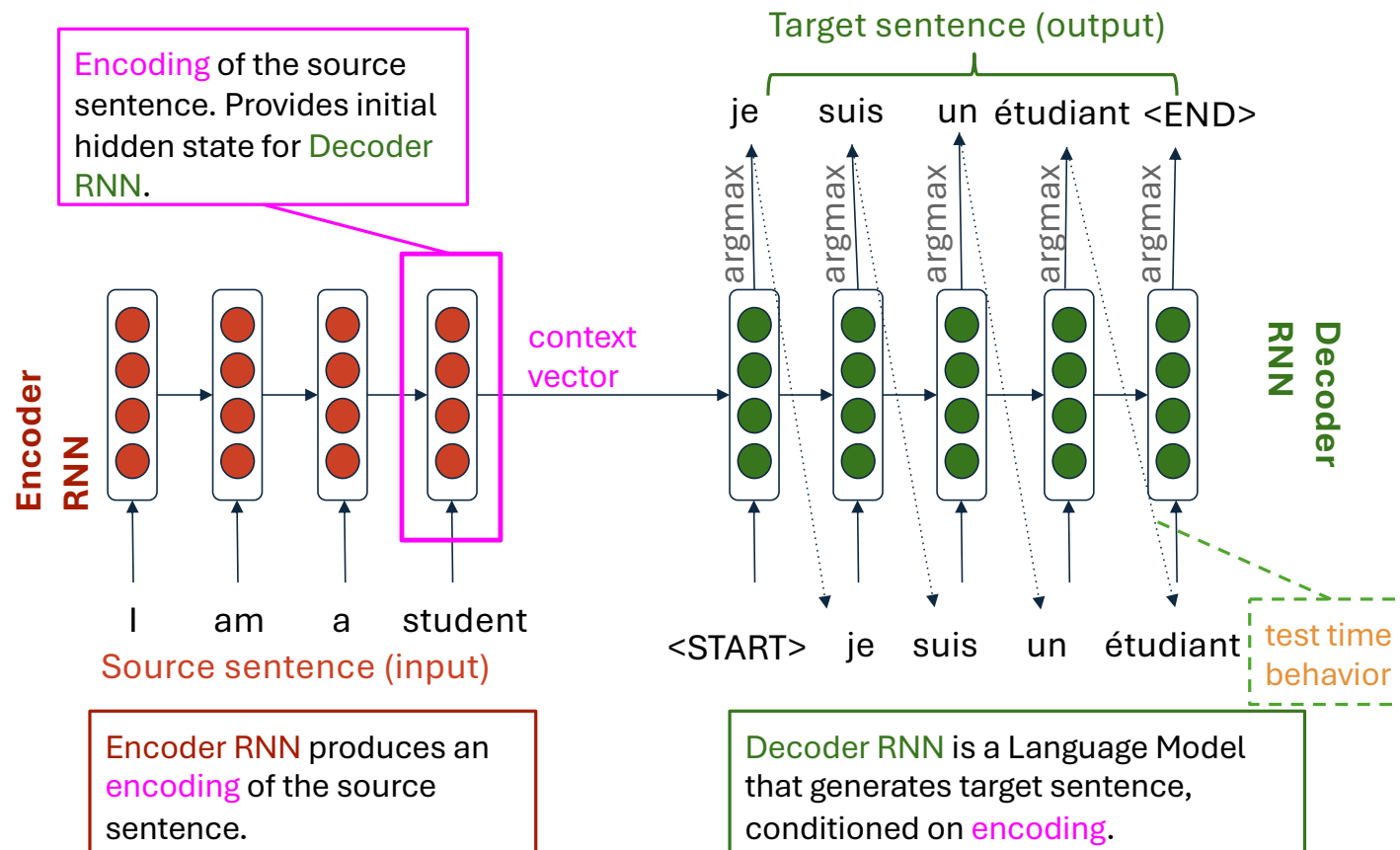
Neural Machine Translation - Seq2Seq



Neural Machine Translation - Seq2Seq



Neural Machine Translation - Seq2Seq



Seq2Seq is versatile!

- Seq2seq is useful for **more than just MT**
- Many NLP tasks can be phrased as sequence-to-sequence:
 - Summarization (long text → short text)
 - Dialogue (previous utterances → next utterance)
 - Code generation (natural language → Python code)

Neural Machine Translation (NMT)

- The seq2seq model is an example of a **Conditional Language Model**.
 - **Language Model** because the decoder is predicting the next word of the **target sentence y**
 - **Conditional** because its predictions are also conditioned on the **source sentence x**

- NMT directly calculates $P(y|x)$:

$$P(y|x) = P(y_1|x) P(y_2|y_1, x) P(y_3|y_1, y_2, x) \dots P(y_T|y_1, \dots, y_{T-1}, x)$$

Probability of next target word,
given target words so far and
source sentence x

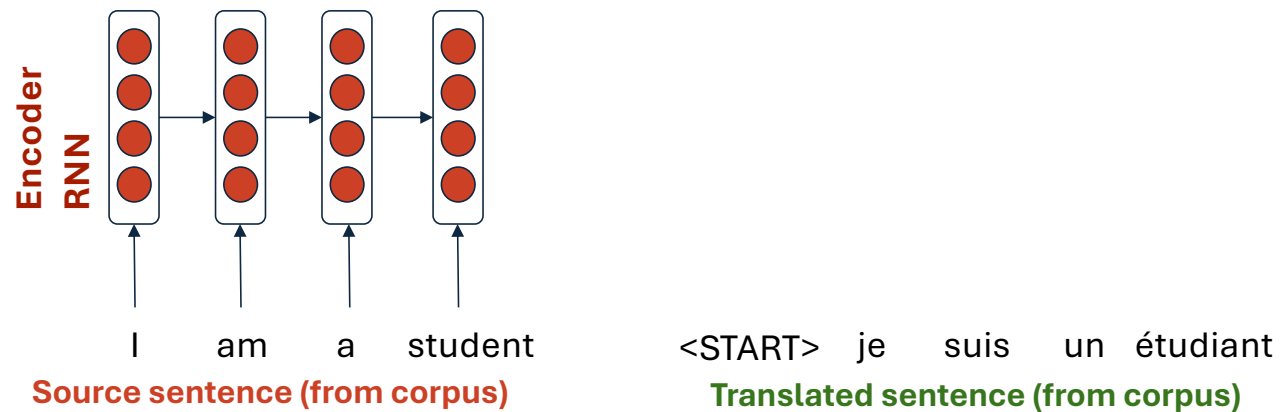
- How to train a NMT system?
 - Easy answer: Get a big parallel corpus...
 - Now: there is some advanced and exciting work.

Training a Neural Machine Translation system

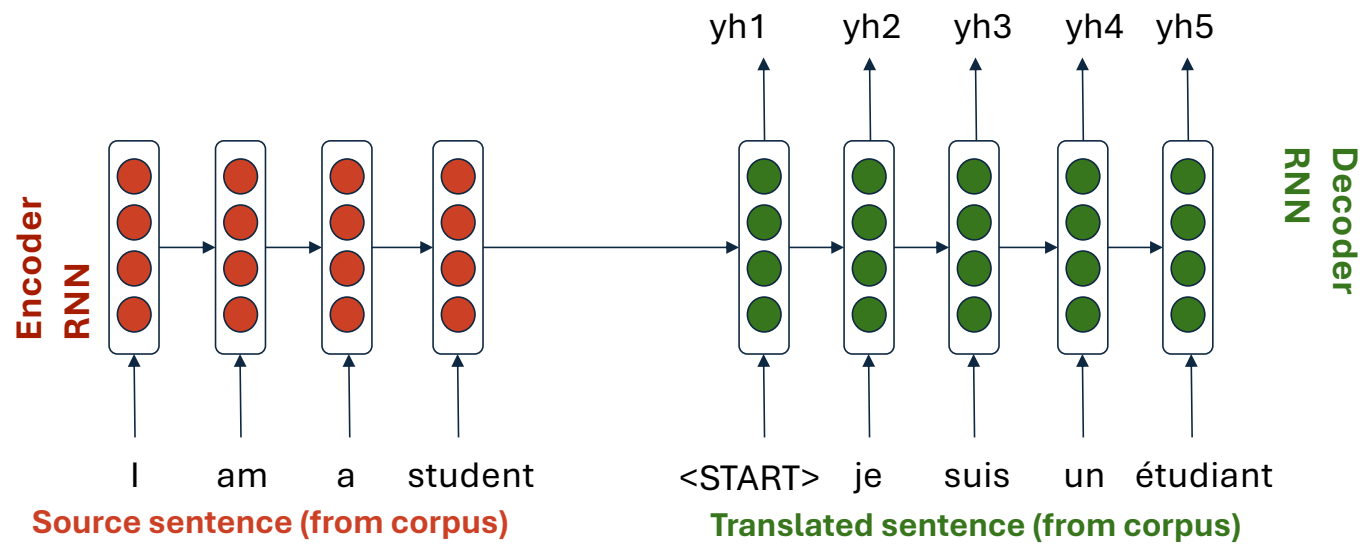
I am a student
Source sentence (from corpus)

<START> je suis un étudiant
Translated sentence (from corpus)

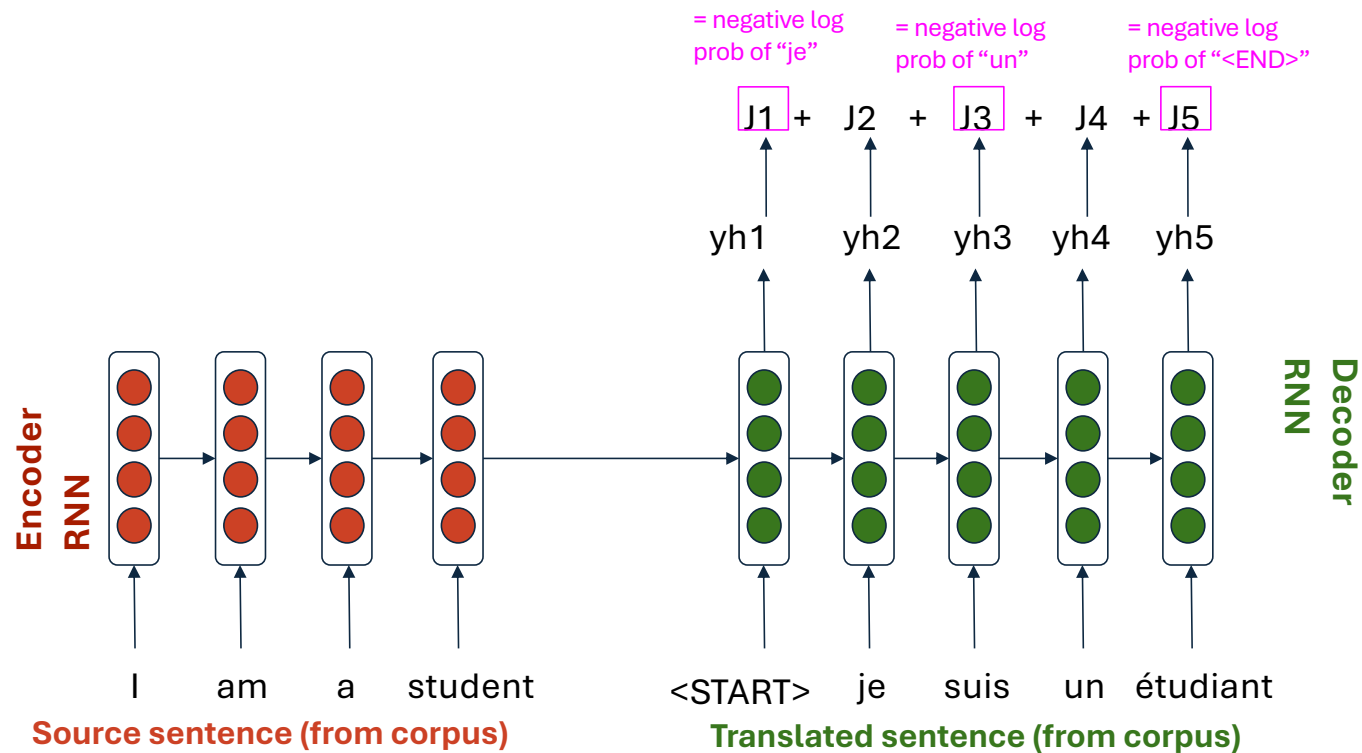
Training a Neural Machine Translation system



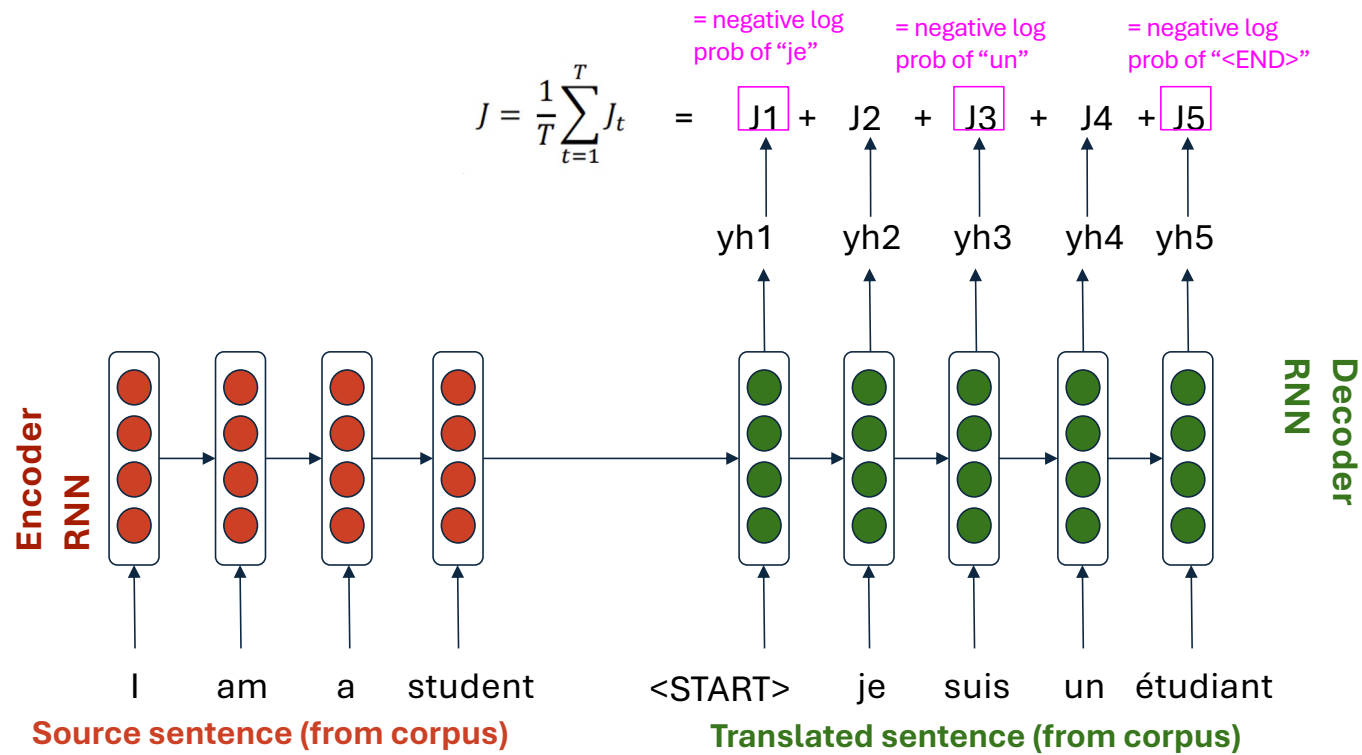
Training a Neural Machine Translation system



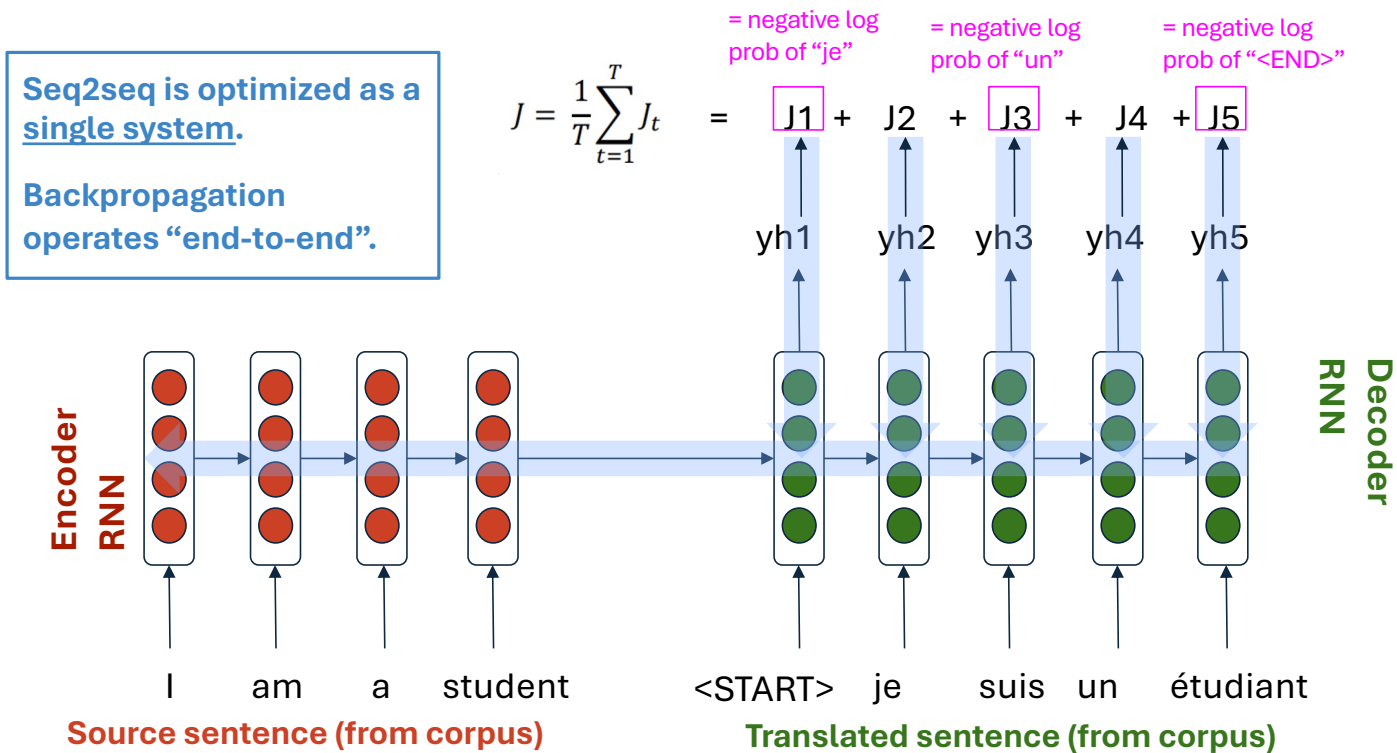
Training a Neural Machine Translation system



Training a Neural Machine Translation system



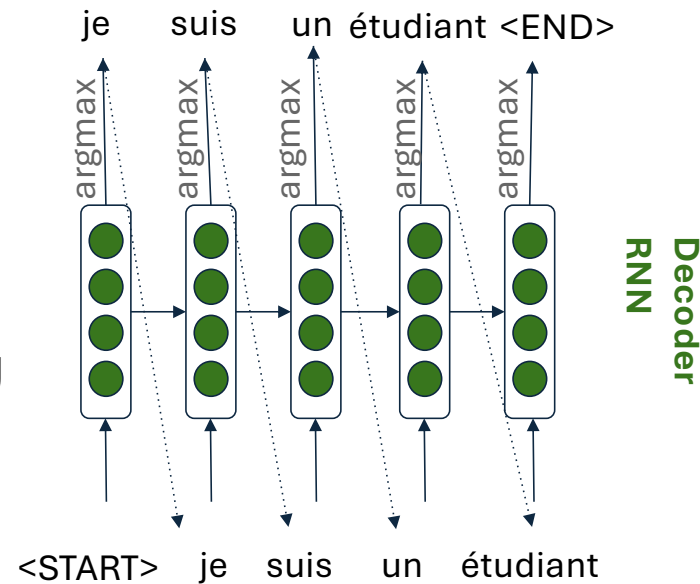
Training a Neural Machine Translation system



Greedy decoding

- We saw how to generate (or “decode”) the target sentence by taking **argmax** on each step of the decoder.
- Greedy decoding: **most probable word** on each step

- Problems?
 - Greedy decoding has no way to undo decisions!
- How to fix this?
 - Exhaustive search decoding
 - Beam search decoding



Exhaustive Search Decoding

- Ideally, we want to find a (length T) translation y that maximizes

$$\begin{aligned} P(y|x) &= P(y_1|x) P(y_2|y_1, x) P(y_3|y_1, y_2, x) \dots, P(y_T|y_1, \dots, y_{T-1}, x) \\ &= \prod_{t=1}^T P(y_t|y_1, \dots, y_{t-1}, x) \end{aligned}$$

- We could try computing all possible sequences y
 - On each step t of the decoder, we're tracking possible partial translations, where V is vocab V^t size
 - This $O(V^T)$ complexity is **far too expensive!**

Beam Search Decoding

- Core idea: **k most probable** partial translations (which we call **hypotheses**)
 - k is the beam size (in practice around 5 to 10)
- A hypothesis $y_1, \dots, y_t$ has a **score** which is its **log probability**:

$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t | x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

- Scores are all negative, and higher score is better
 - Search for high-scoring hypotheses, tracking top k on each step
- Beam search is **not guaranteed** to find optimal solution
 - But **much more efficient** than exhaustive search!

Beam Search Decoding: example

Beam size $k = 2$. Blue numbers

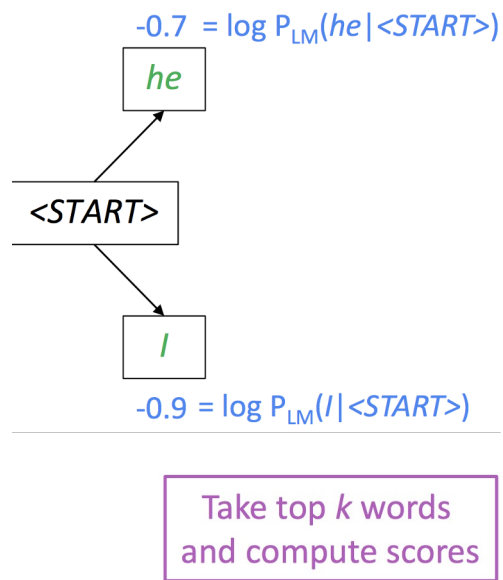
$$\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

<START>

Calculate prob
dist of next word

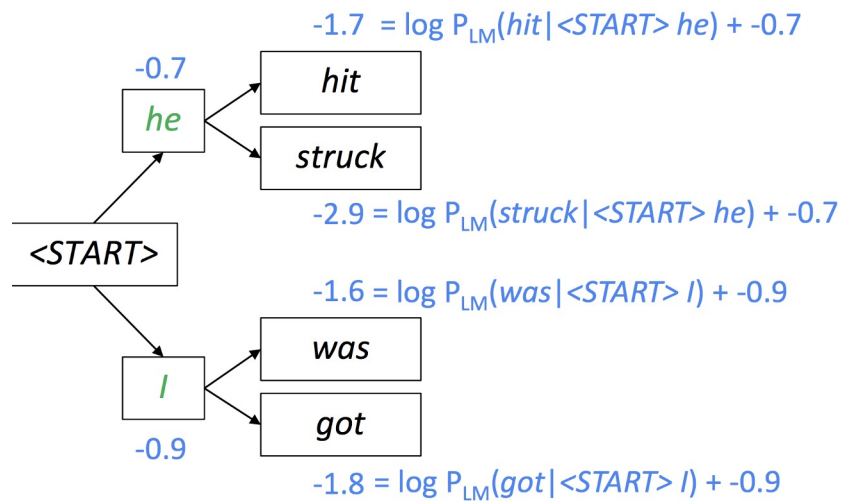
Beam Search Decoding: example

Beam size $k = 2$. Blue numbers $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



Beam Search Decoding: example

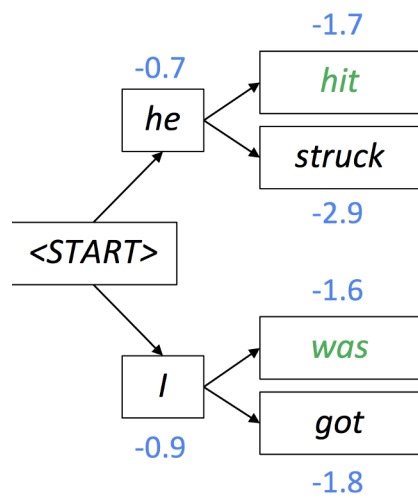
Beam size $k = 2$. Blue numbers $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



For each of the k hypotheses, find top k next words and calculate scores

Beam Search Decoding: example

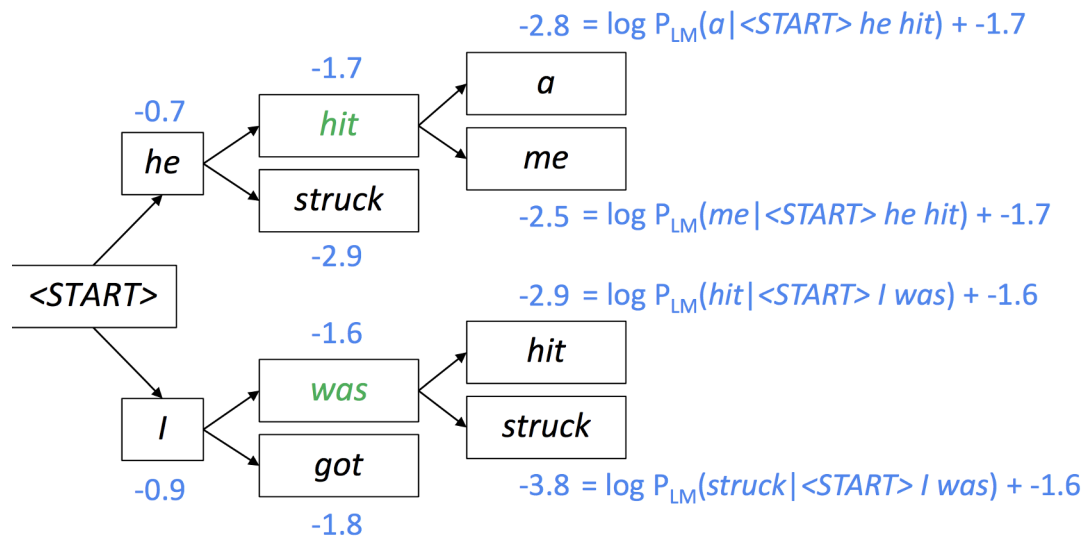
Beam size $k = 2$. Blue numbers $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



Of these k^2 hypotheses,
just keep k with highest scores

Beam Search Decoding: example

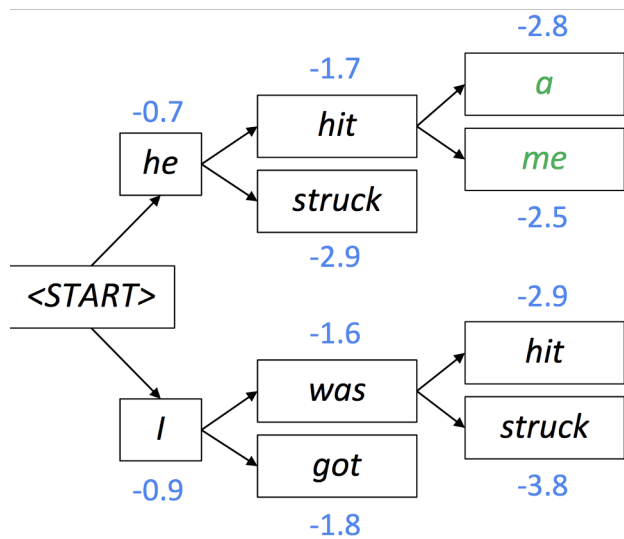
Beam size $k = 2$. Blue numbers $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



For each of the k hypotheses, find top k next words and calculate scores

Beam Search Decoding: example

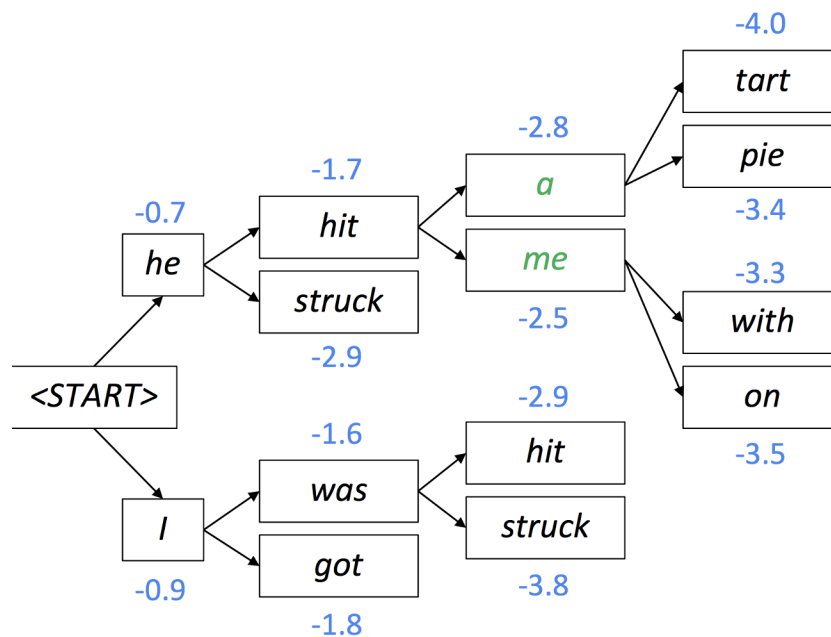
Beam size $k = 2$. Blue numbers $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



Of these k^2 hypotheses,
just keep k with highest scores

Beam Search Decoding: example

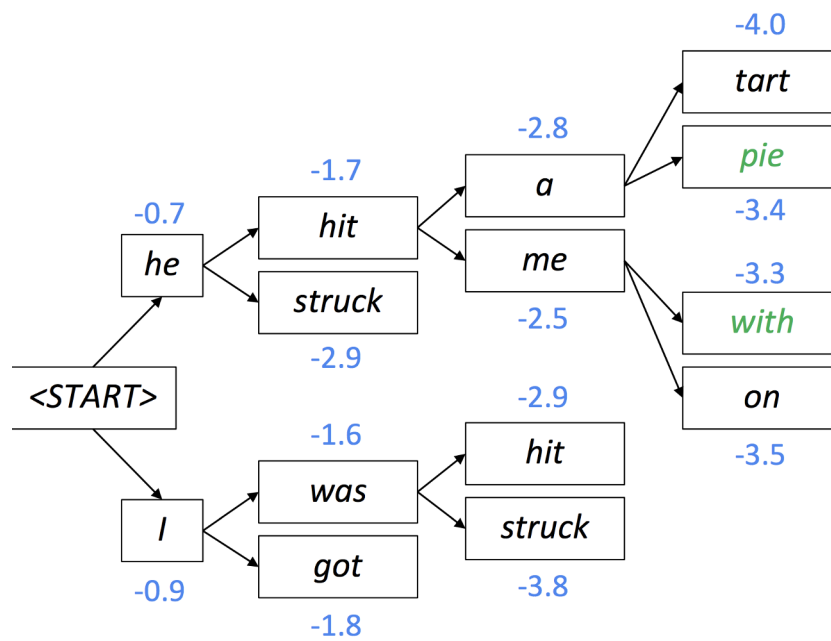
Beam size $k = 2$. Blue numbers $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



For each of the k hypotheses, find top k next words and calculate scores

Beam Search Decoding: example

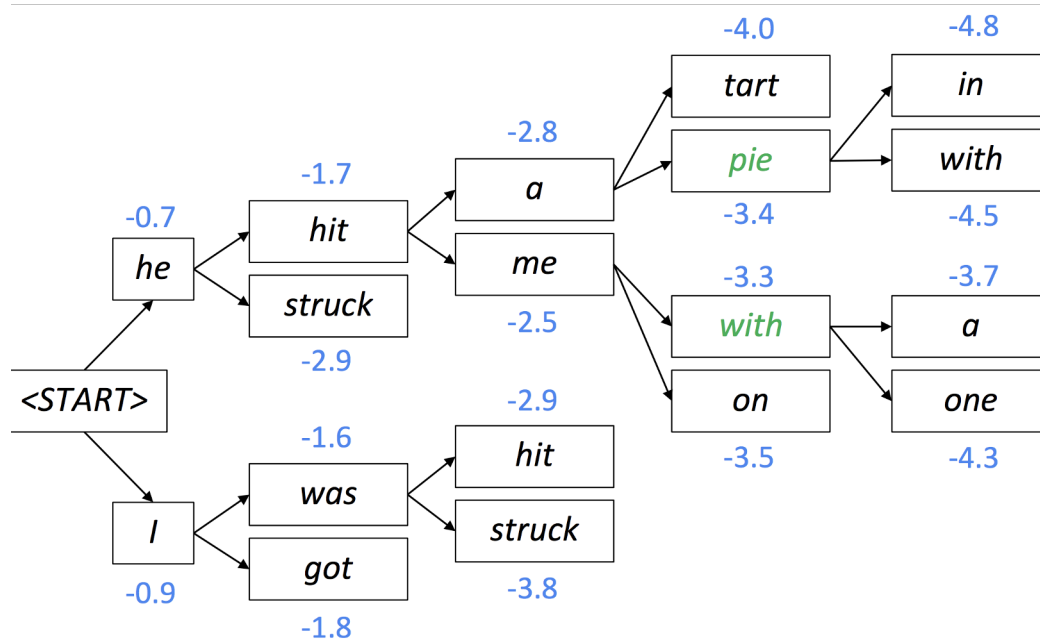
Beam size $k = 2$. Blue numbers $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



Of these k^2 hypotheses,
just keep k with highest scores

Beam Search Decoding: example

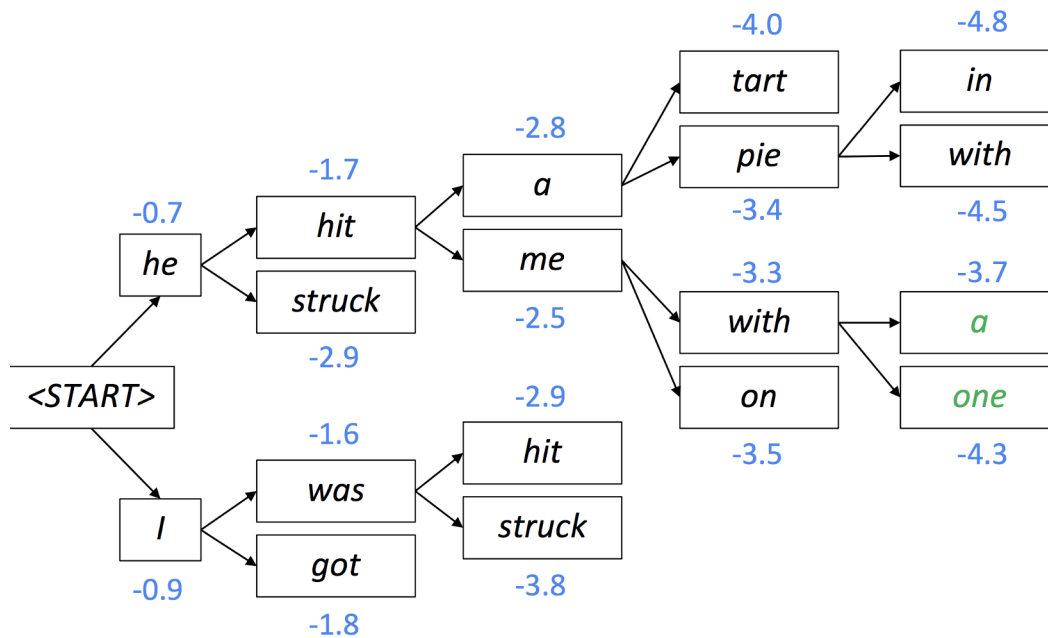
Beam size $k = 2$. **Blue numbers** $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



For each of the k hypotheses, find top k next words and calculate scores

Beam Search Decoding: example

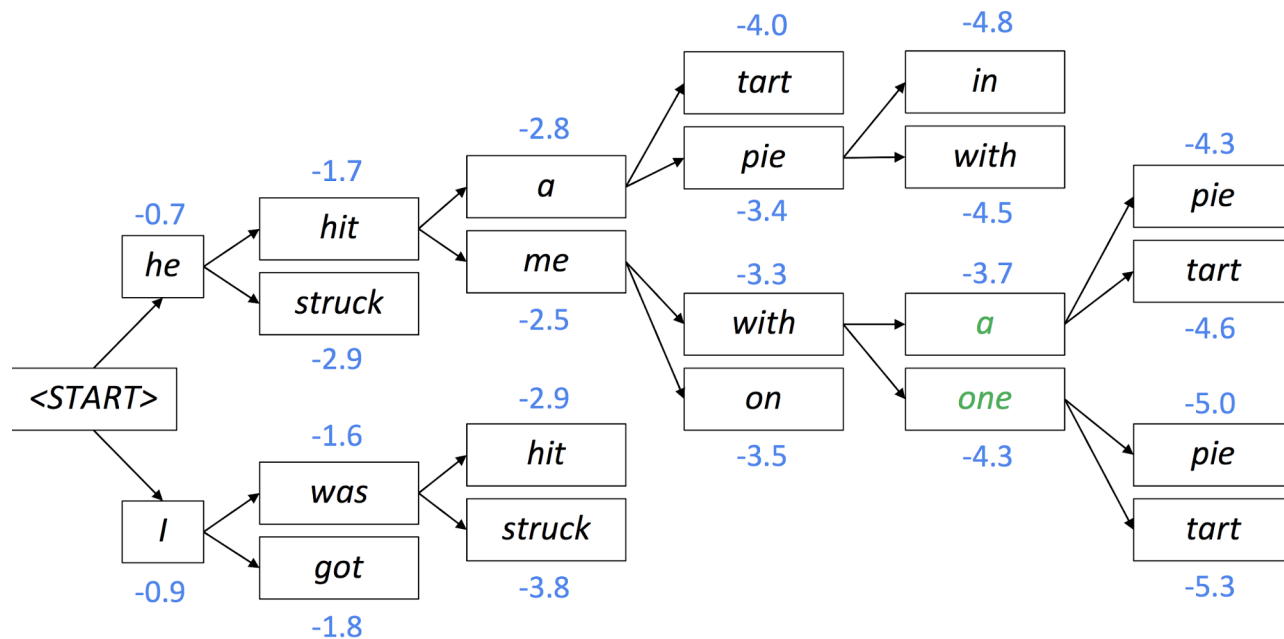
Beam size $k = 2$. Blue numbers $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



Of these k^2 hypotheses,
just keep k with highest scores

Beam Search Decoding: example

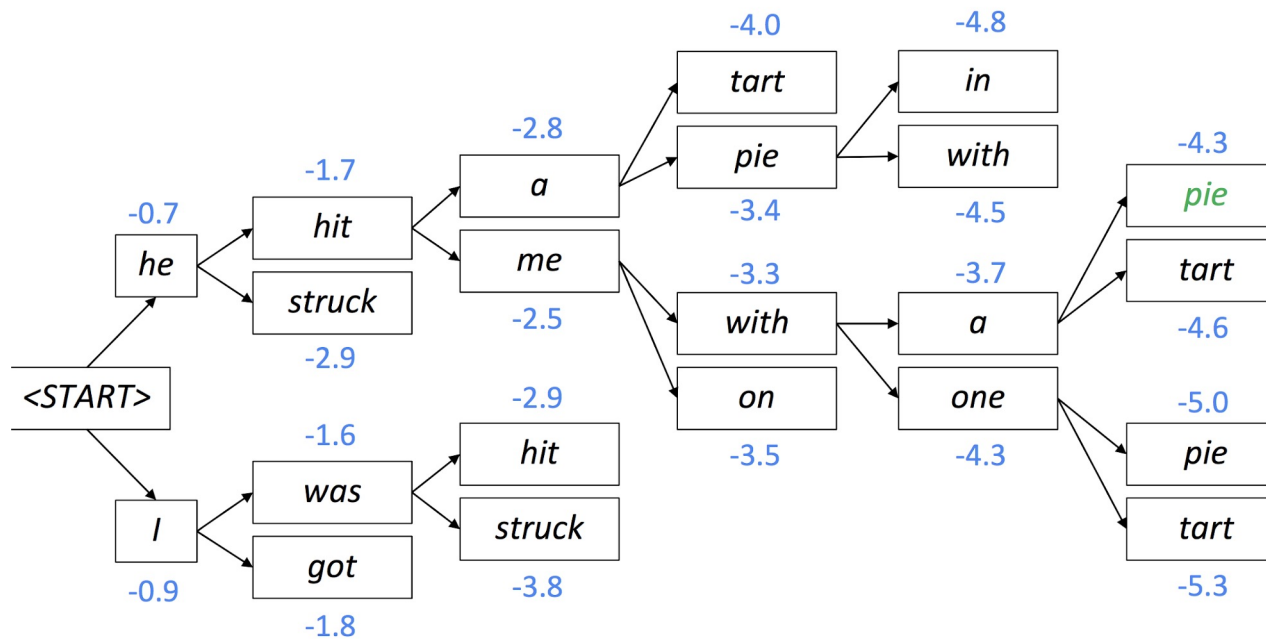
Beam size $k = 2$. **Blue numbers** $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



For each of the k hypotheses, find top k next words and calculate scores

Beam Search Decoding: example

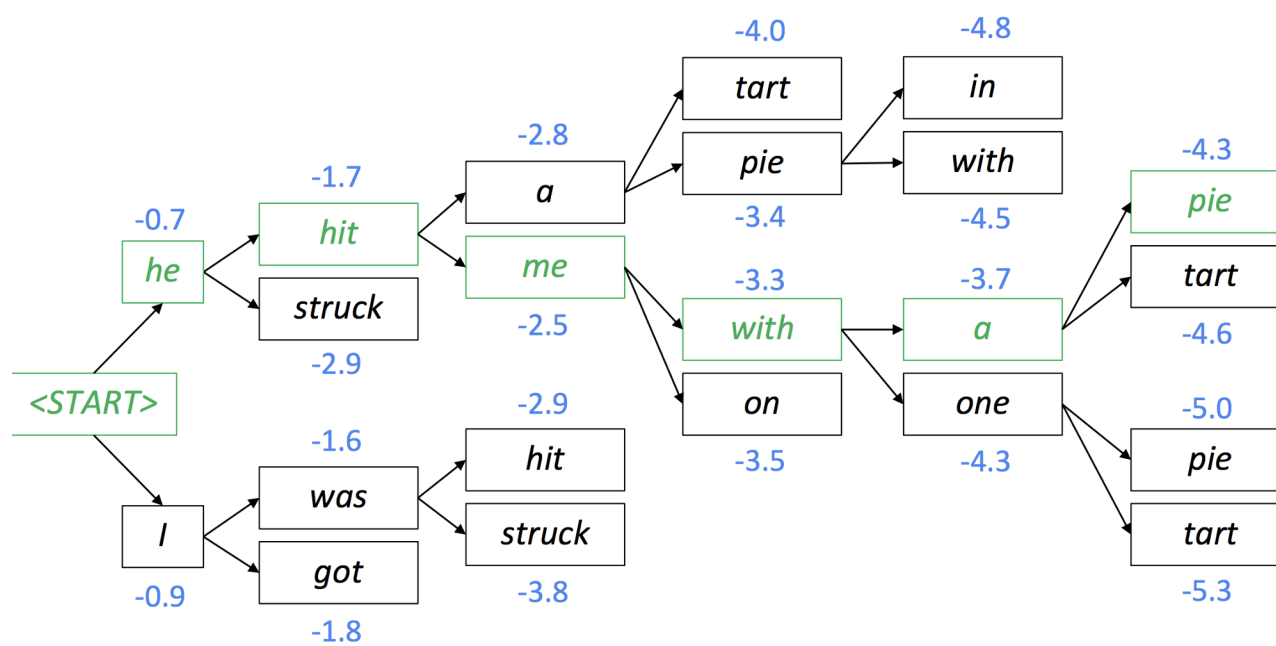
Beam size $k = 2$. Blue numbers $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



This is the top-scoring hypothesis!

Beam Search Decoding: example

Beam size $k = 2$. Blue numbers $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



Backtrack to obtain the full hypothesis

Beam Search Decoding: stopping criterion

- In **greedy decoding**, usually we decode until the model produces a <END> token
 - e.g. <START> he hit me with a pie <END>
- In **beam search decoding**, different hypotheses may produce <END> tokens on **different timesteps**
 - When a hypothesis produces <END>, that hypothesis is complete
 - **Place it aside** and continue exploring other hypotheses via beam search.
- Usually we continue beam search until:
 - We reach **timestep T** (where T is some pre-defined cutoff), or
 - We have **at least n completed hypotheses** (where n is pre-defined cutoff)

Beam Search Decoding: finishing up

- We have our list of completed hypotheses.
- How to select top one with highest score?
- Each hypothesis $y_1, \dots, y_t$ on our list has a score

$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t | x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

- Problem: longer hypotheses have lower scores
- Fix: Normalize by length. Use this to select top one instead:

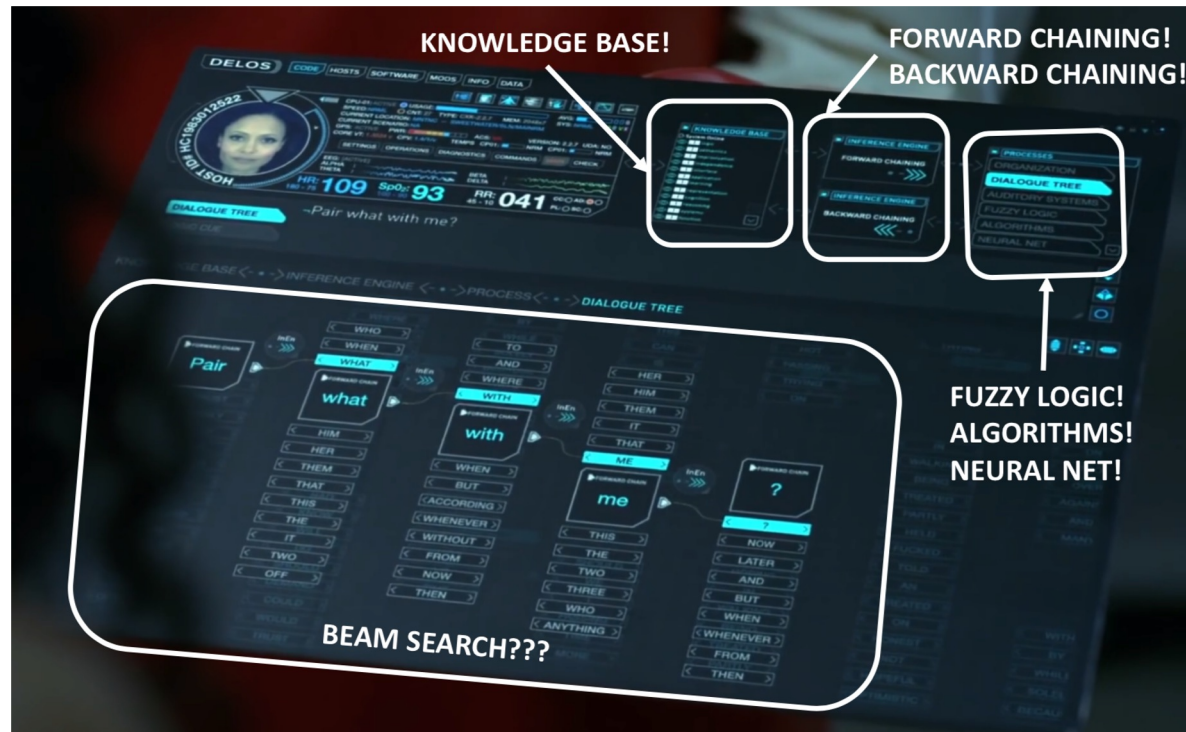
$$\frac{1}{t} \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

Beam Search Decoding

Beam search decoding helps in producing **more diverse, fluent, and accurate** results compared to simple greedy decoding.

- **Diverse Output Generation**
- **Global Context**
- **Handling Variable Length Outputs**

Aside: do the hosts in Westworld use beam search?



Source: <https://www.youtube.com/watch?v=ZnxJRYit44k>

What's the effect of changing beam size k ?

- **Small k** (greedy decoding: $k=1$)
 - Ungrammatical, unnatural, nonsensical, incorrect
- **Larger k** means you consider more hypotheses
 - Increasing k reduces some of the problems above
 - Larger k is more computationally expensive
 - Increase k decreases BLEU score (Tu *et al*, Koehn *et al*)
 - In open-ended tasks like chit-chat dialogue, large k can make output more generic (see next slide)

Effect of beam size in chitchat dialogue

I mostly eat a fresh
and raw diet, so I
save on groceries

**Human
chit-chat
partner**

Beam size	Model response
1	<i>I love to eat healthy and eat healthy</i>
2	<i>That is a good thing to have</i>
3	<i>I am a nurse so I do not eat raw food</i>
4	<i>I am a nurse so I am a nurse</i>
5	<i>Do you have any hobbies?</i>
6	<i>What do you do for a living?</i>
7	<i>What do you do for a living?</i>
8	<i>What do you do for a living?</i>

Low beam size:

More on-topic but
nonsensical;
bad english

High beam size:

Converges to safe,
“correct” response,
but it’s generic and
less relevant

Sampling-based decoding

- **Pure sampling**

- On each step t , randomly sample from the probability distribution P_t to obtain your next word
- Like greedy decoding, but sample instead of argmax

- **Top-n sampling***

- On each step t , randomly sample from P_t , **restricted to just the top-n most probable words**
- Like pure sampling, but truncate the probability distribution
- $n=1$ is greedy search, $n=V$ is pure sampling
- **Increase** n to get more **diverse/risky** output
- **Decrease** n to get more **generic/safe** output

Softmax temperature

- **Recall:** on time step t , the LM computes a prob. dist P_t by applying the softmax function to a vector of scores $s \in \mathbb{R}^{|V|}$

$$P_t(w) = \frac{\exp(s_w)}{\sum_{u \in V} \exp(s_u)}$$

- You can apply a temperature hyperparameter to the softmax:

$$P_t(w) = \frac{\exp(s_w/\tau)}{\sum_{u \in V} \exp(s_u/\tau)}$$

- **Raise the temperature τ :** P_t becomes more **uniform**
 - Thus **more diverse** output (probability is spread around vocab)
- **Lower the temperature τ :** P_t becomes more **spiky**
 - Thus **less diverse** output (probability is concentrated on top words)

Note: softmax temperature is not a decoding algorithm!
It's a technique you can apply at test time, in conjunction with a decoding algorithm (such as beam search or sampling)

Decoding algorithms: in summary

- Greedy decoding
- Beam search
- Sampling methods
 - Good for open-ended / creative generation (poetry, stories)
 - Top-n sampling allows you to control diversity
- Softmax temperature
 - Not a decoding algorithm!

Advantages of NMT

- Compared to SMT, NMT has many advantages:
 - Better **performance**
Fluent, better use of context, phrase similarities
 - A **single neural network** to be optimized end-to-end
No subcomponents individually optimized
 - Requires much **less human engineering effort**
 - No feature engineering
 - Same method for all language paris

Disadvantages of NMT

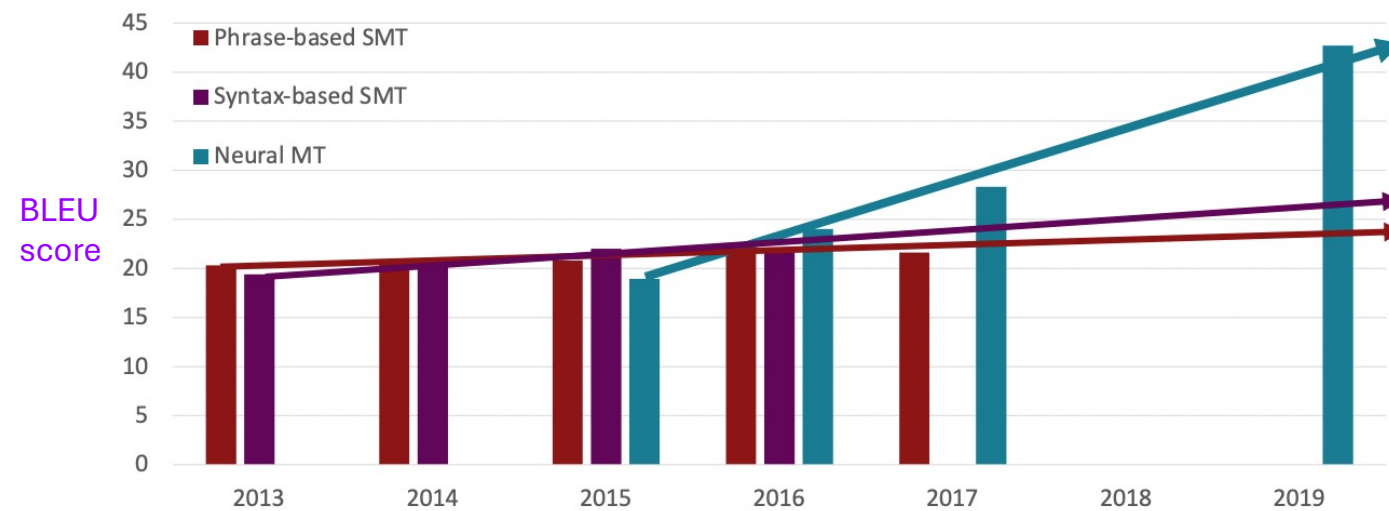
- Compared to SMT:
 - NMT is less interpretable
 - NMT is difficult to control

How do we evaluate Machine Translation?

- BLEU (**B**ilingual **E**valuation **U**nderstudy)
 - **n-gram precision** (usually for 1, 2, 3 and 4-grams)
 - Plus a penalty for too-short system translations
 - Higher BLEU scores indicate better translation quality.
- BLEU is **useful** but **imperfect**
 - Variety of translation
 - Good translation can get a poor BLEU score
 - Can be used combined with human evaluation.

MT progress over time

[Edinburgh En-De WMT newstest2013 Cased BLEU; NMT 2015 from U. Montréal; NMT 2019 FAIR on newstest2019]



source: <https://web.stanford.edu/class/cs224n/slides/cs224n-2023-lecture06-fancy-rnn.pdf>

So is Machine Translation solved?

- Nope!
- Many difficulties remain:
 - Out-of-vocabulary words
 - Domain mismatch between train and test data
 - Maintaining context over longer text
 - Low-resource language pairs

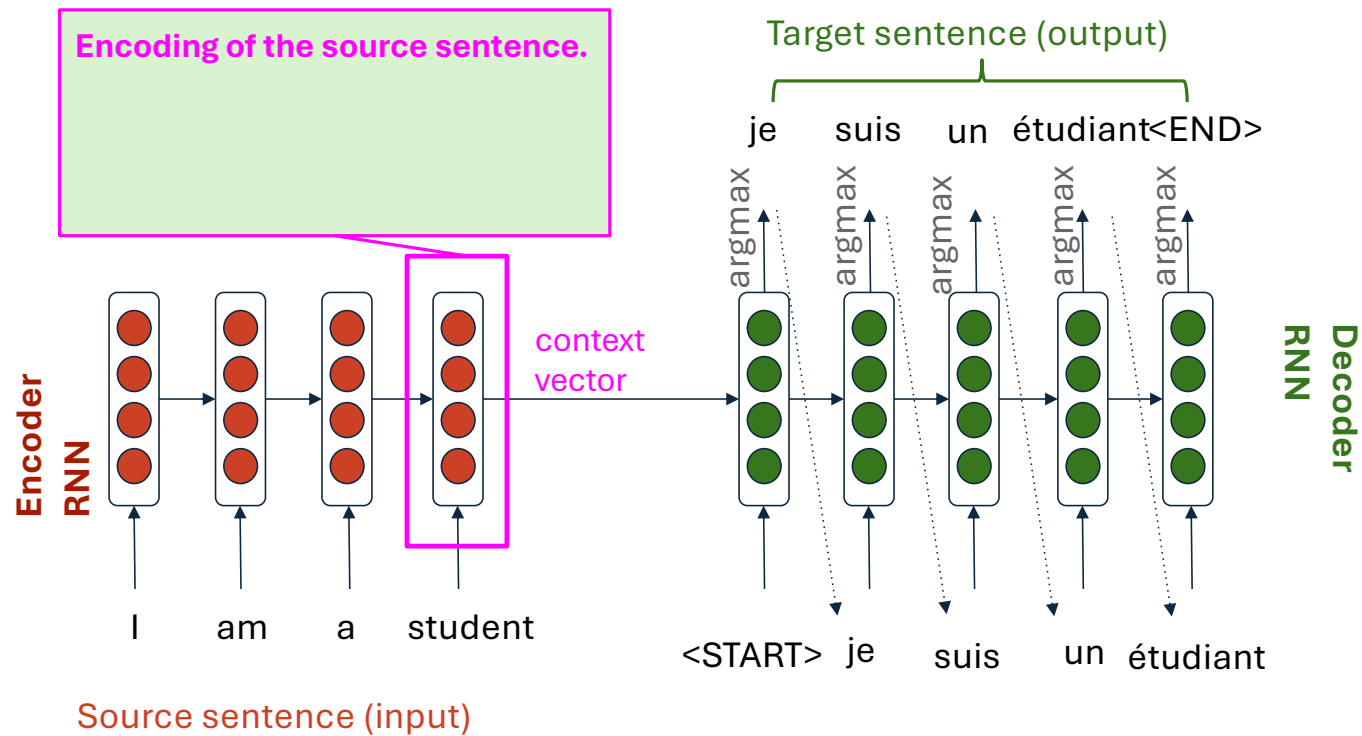
Further reading: [“Has AI surpassed humans at translation? Not even close!”](#)

NMT research continues

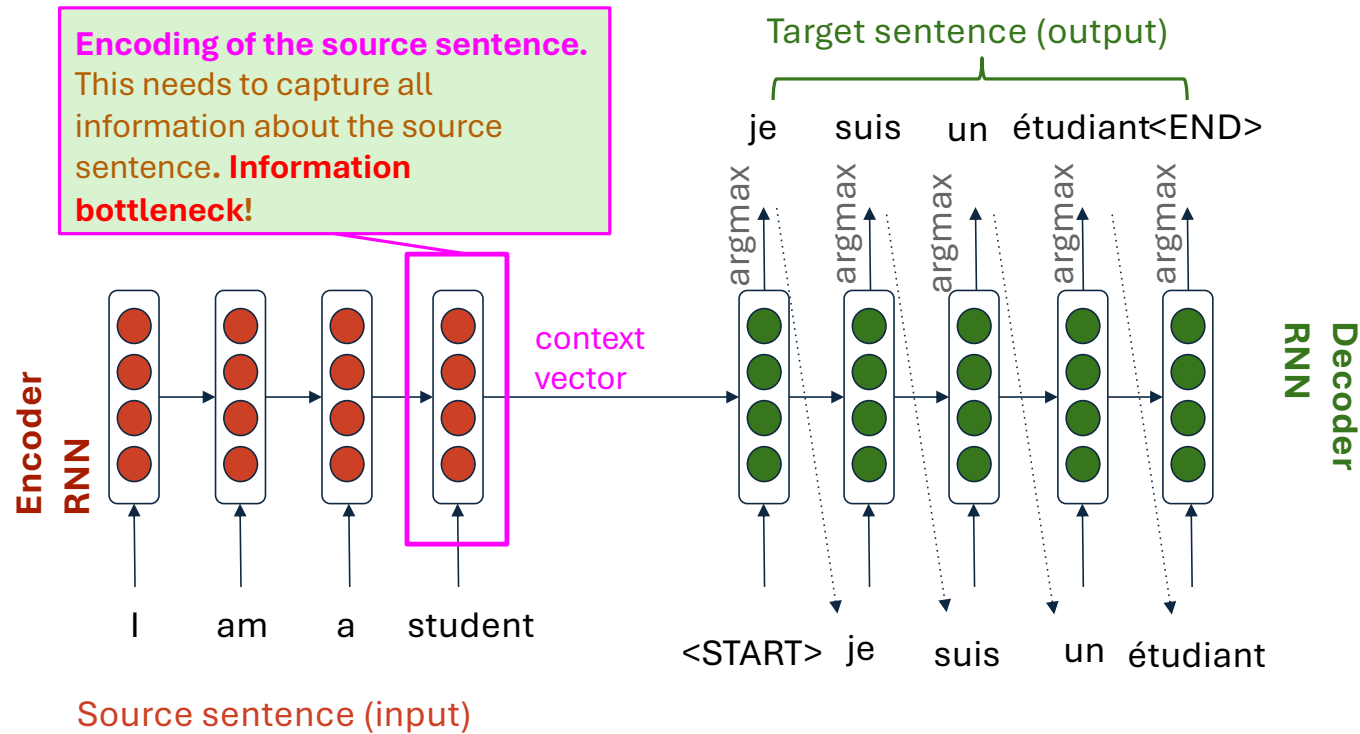
- NMT is the flagship task for NLP Deep Learning
- NMT research has pioneered many of the recent innovations of NLP Deep Learning
- In 2019: NMT research continues to thrive
 - Researchers have found many, many improvements to the “vanilla” seq2seq NMT system we’ve presented today
 - But **one improvement** is so integral that it is the new vanilla...
- We will discuss next:

Attention

Seq2Seq: the bottleneck problem



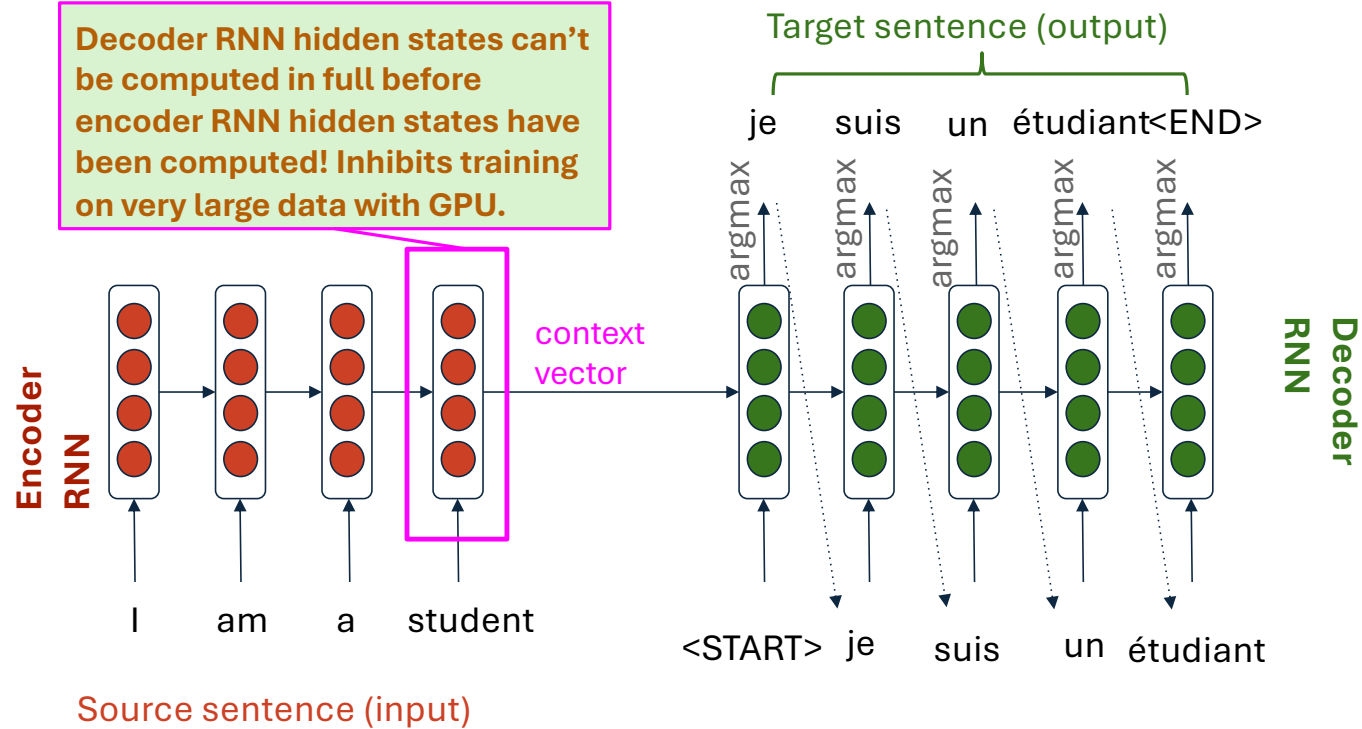
Seq2Seq: the bottleneck problem



Problems with this architecture?

Seq2Seq: the bottleneck problem

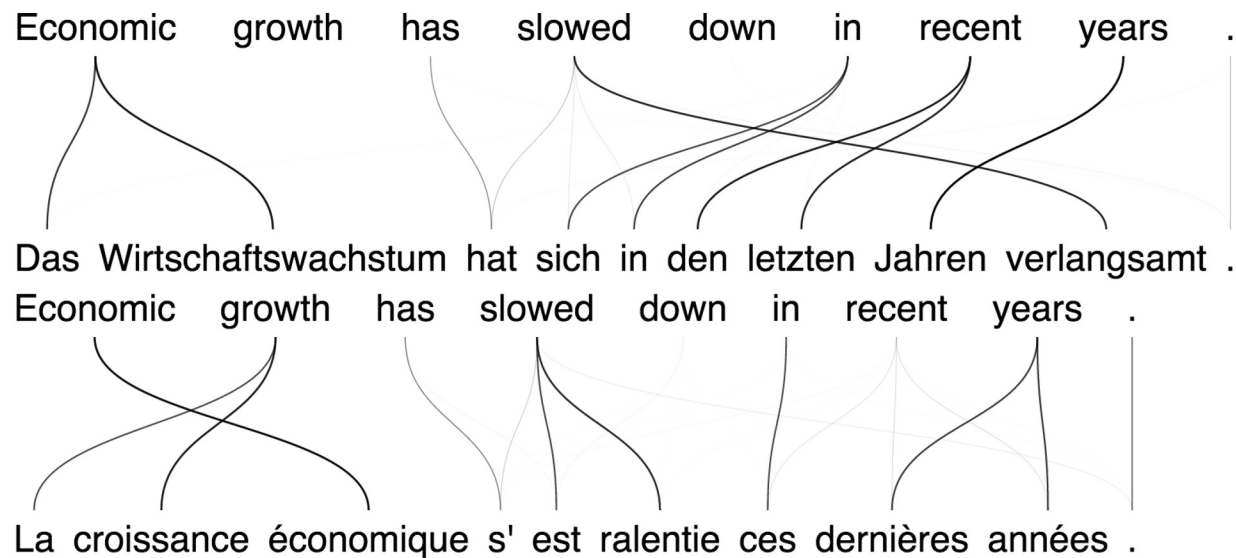
Decoder RNN hidden states can't be computed in full before encoder RNN hidden states have been computed! Inhibits training on very large data with GPU.



Problems with this architecture?

How about attention?

- Attention treats each word's representation as a **query** to access and incorporate information from a set of values.

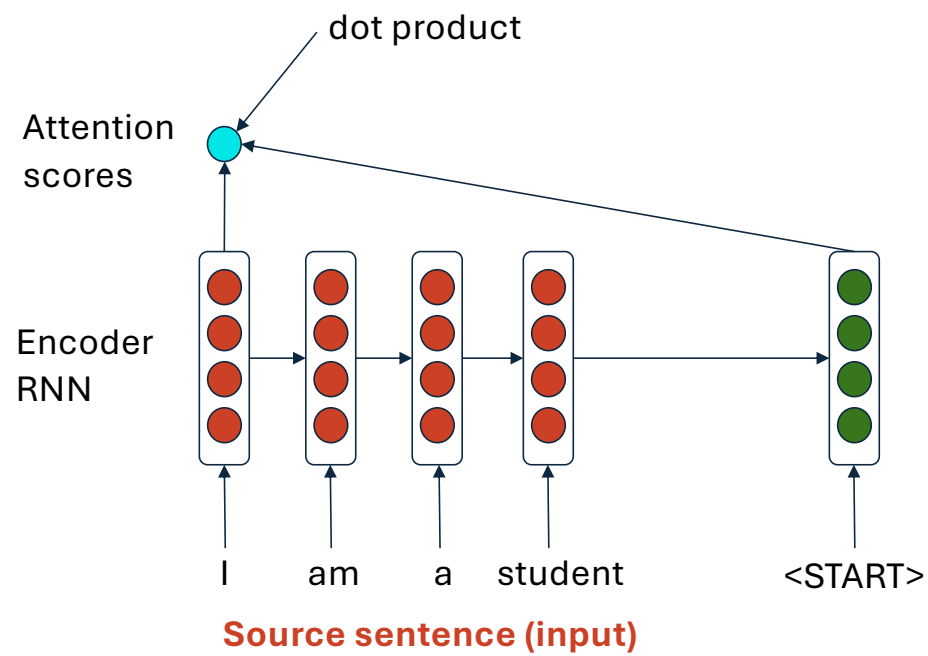


Attention

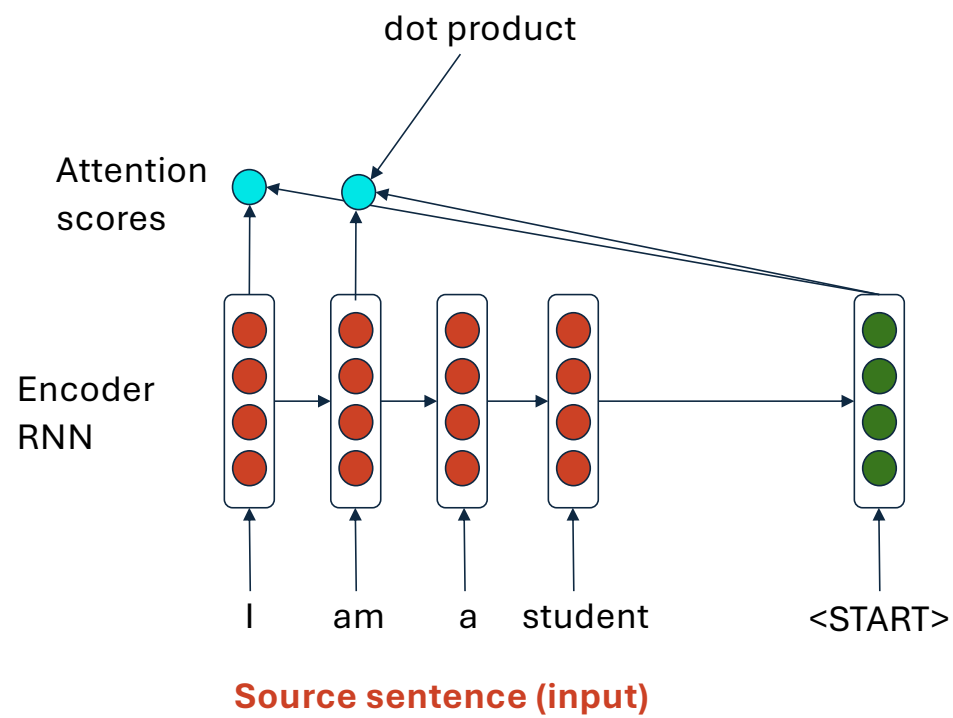
- Attention provides a solution to the bottleneck problem.
- **Core idea:** on each step of the decoder, use **direct connection** to the encoder to **focus on a particular part** of the source sequence.
- There are many types of attention. We'll examine the encoder mechanisms introduced by **Huong et al.[1]**.
- First, we will show via diagram (no equations), then we will show with equations.

[1]. Huong et al. 2015, Effective Approaches to Attention-based Neural Machine Translation.

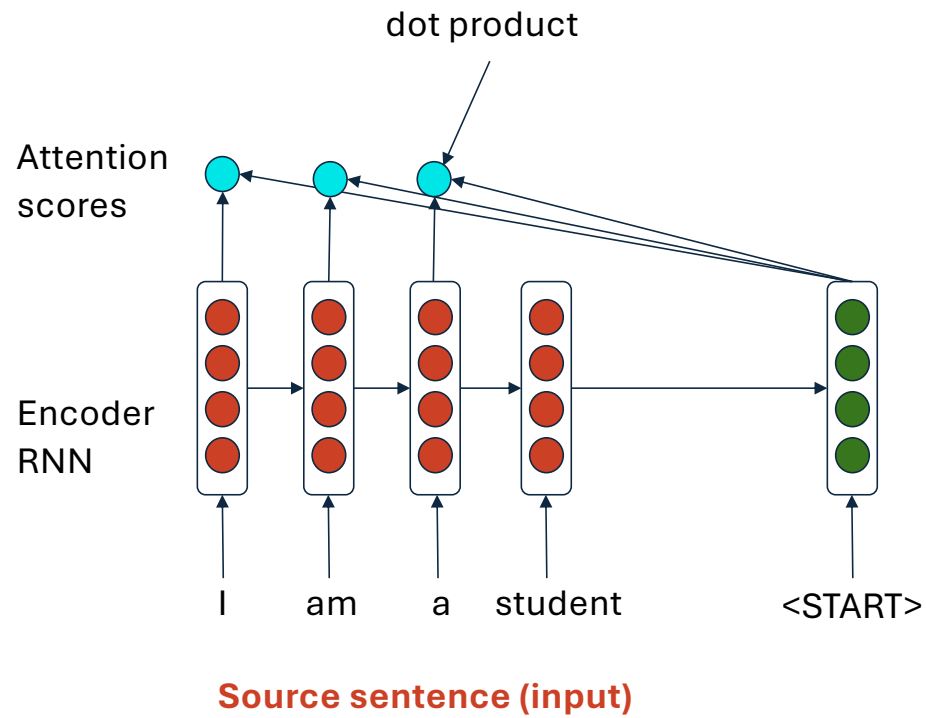
Seq2Seq with attention



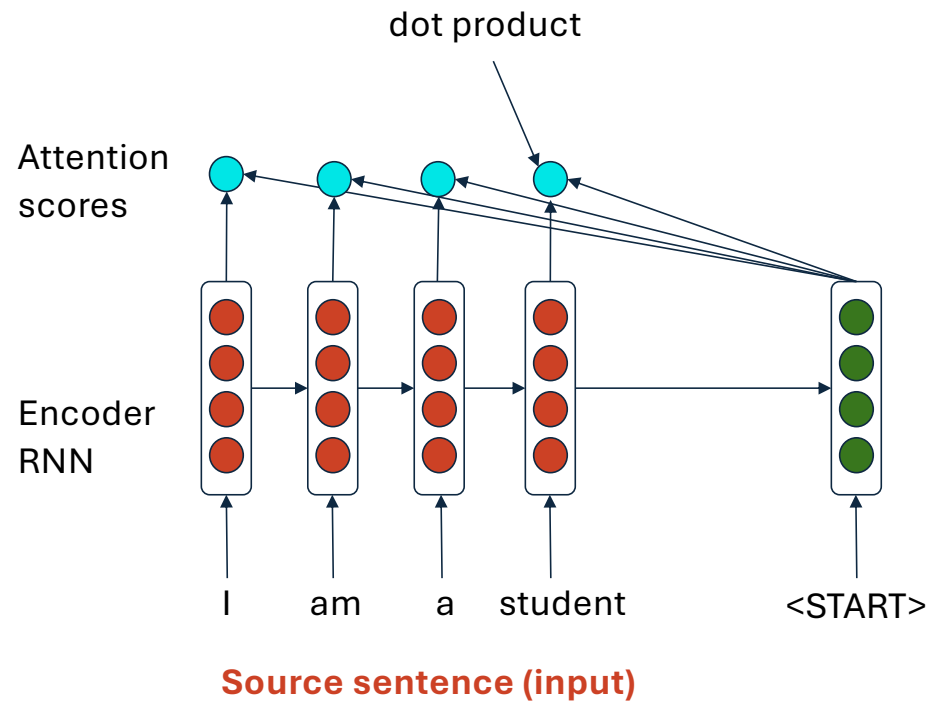
Seq2Seq with attention



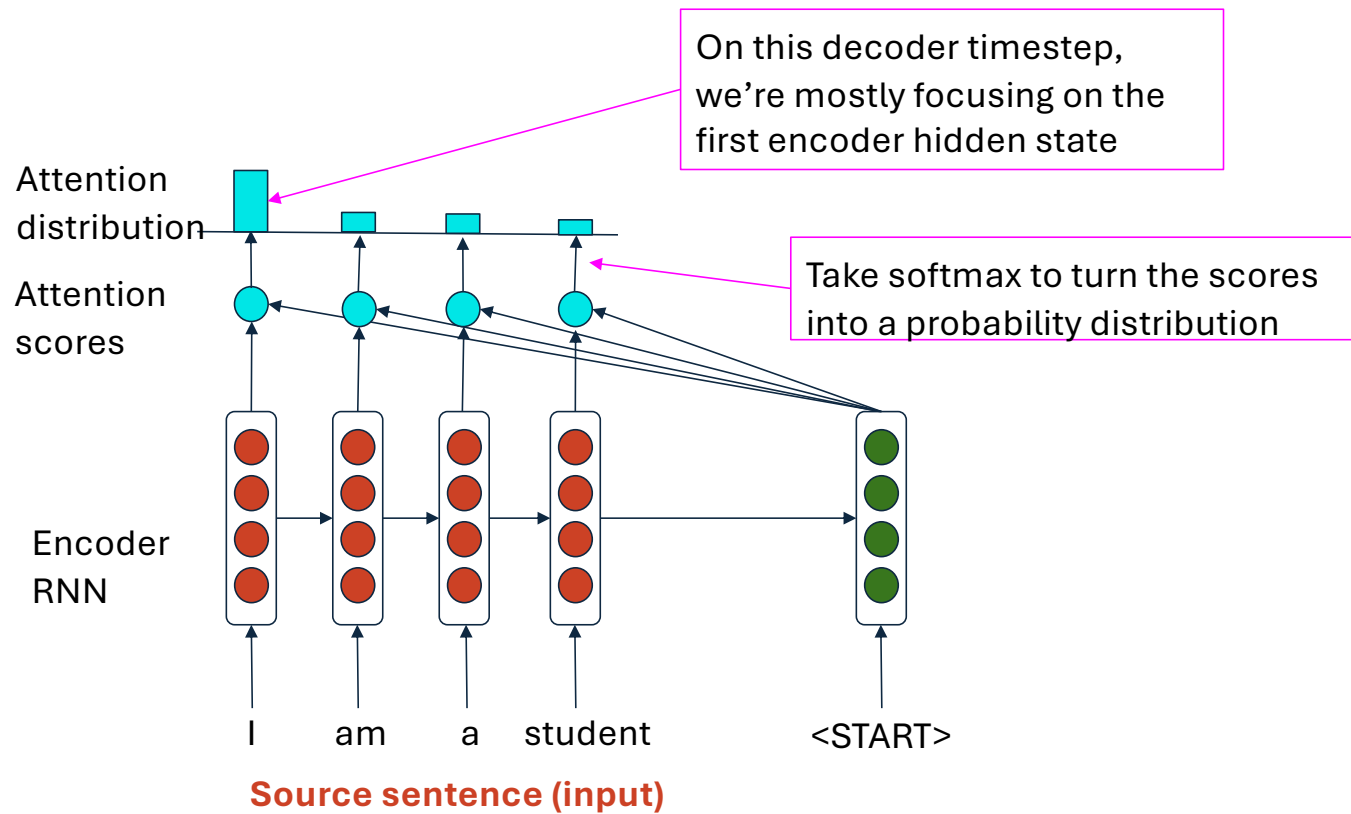
Seq2Seq with attention



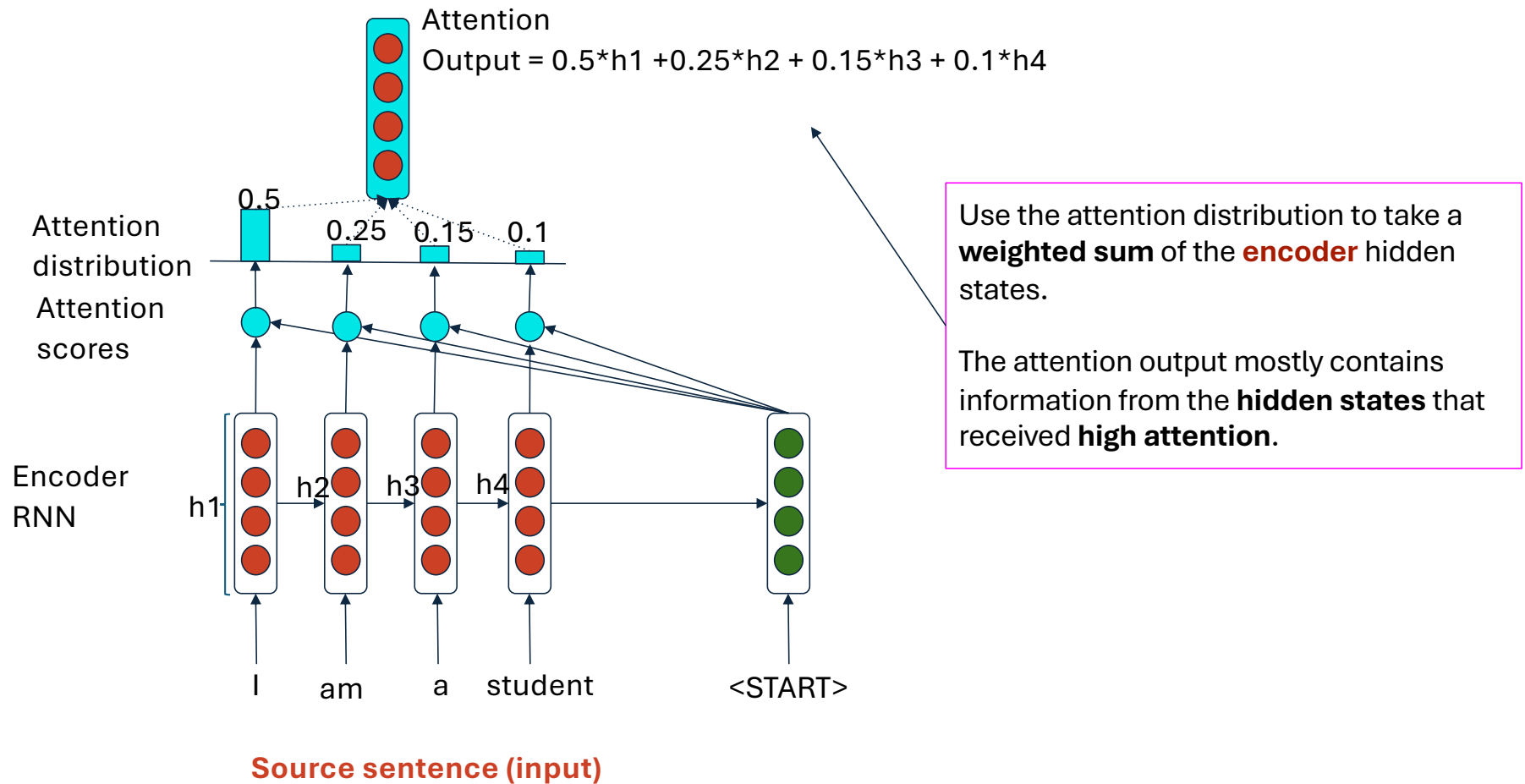
Seq2Seq with attention



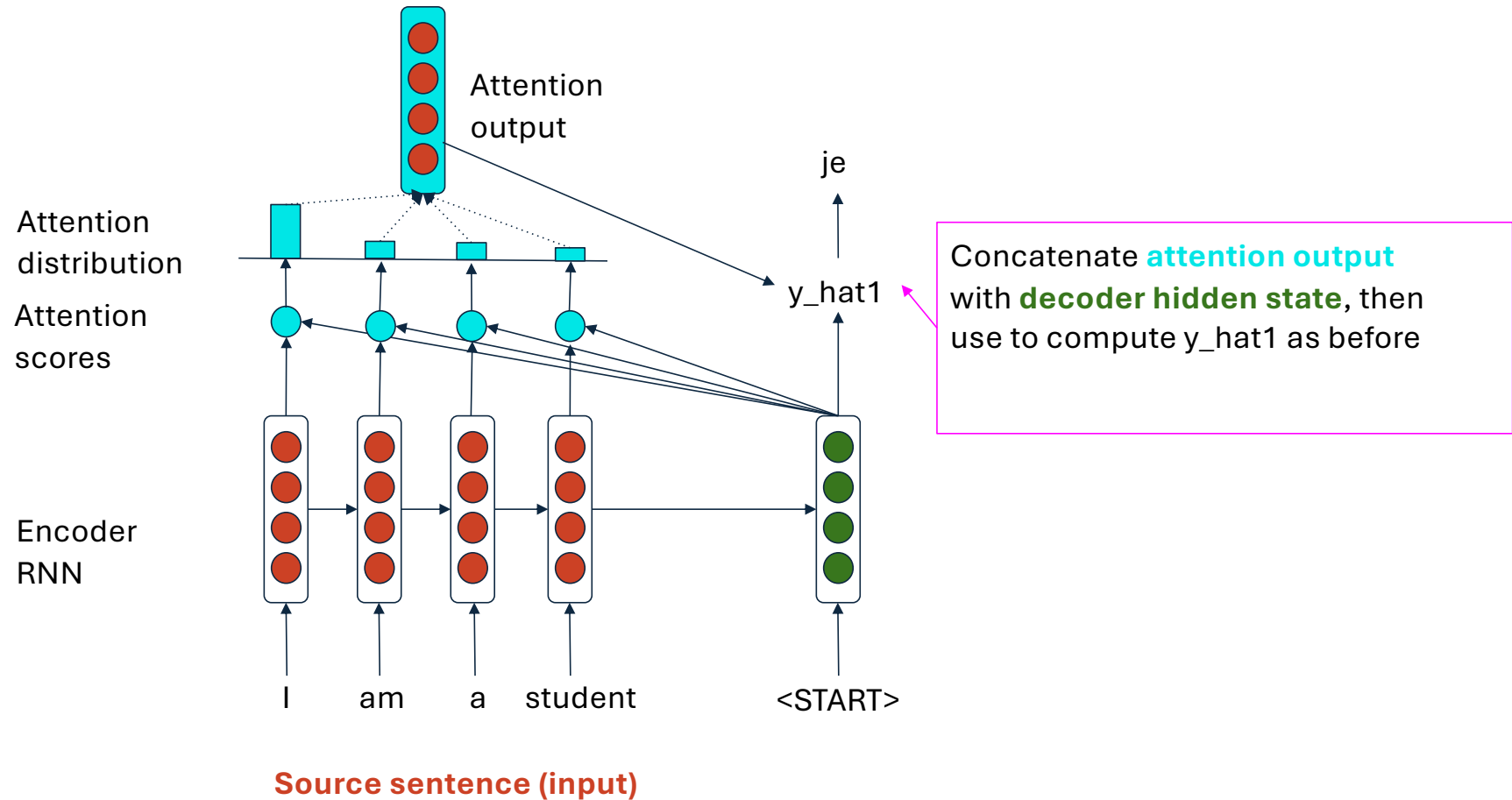
Seq2Seq with attention



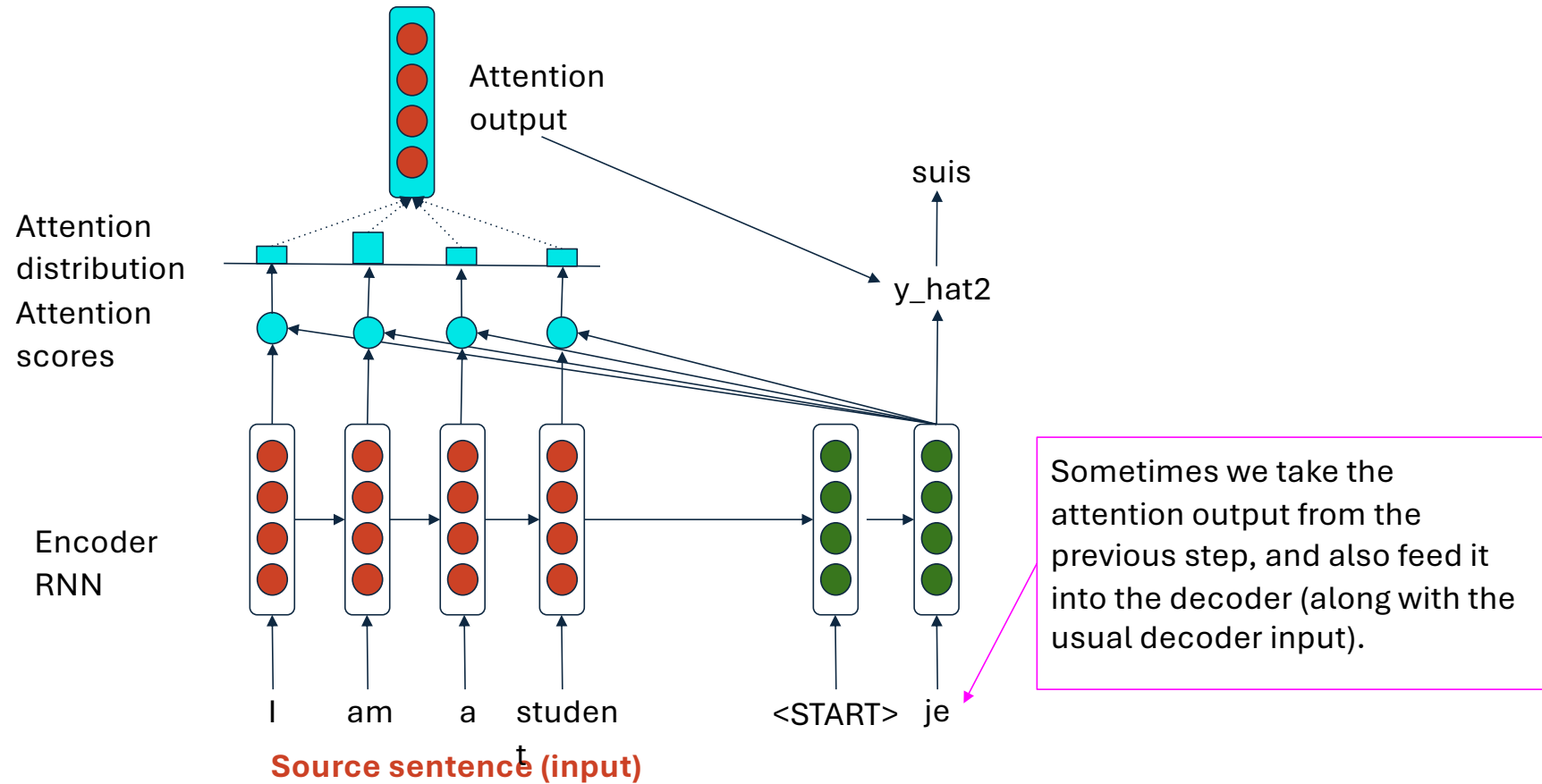
Seq2Seq with attention



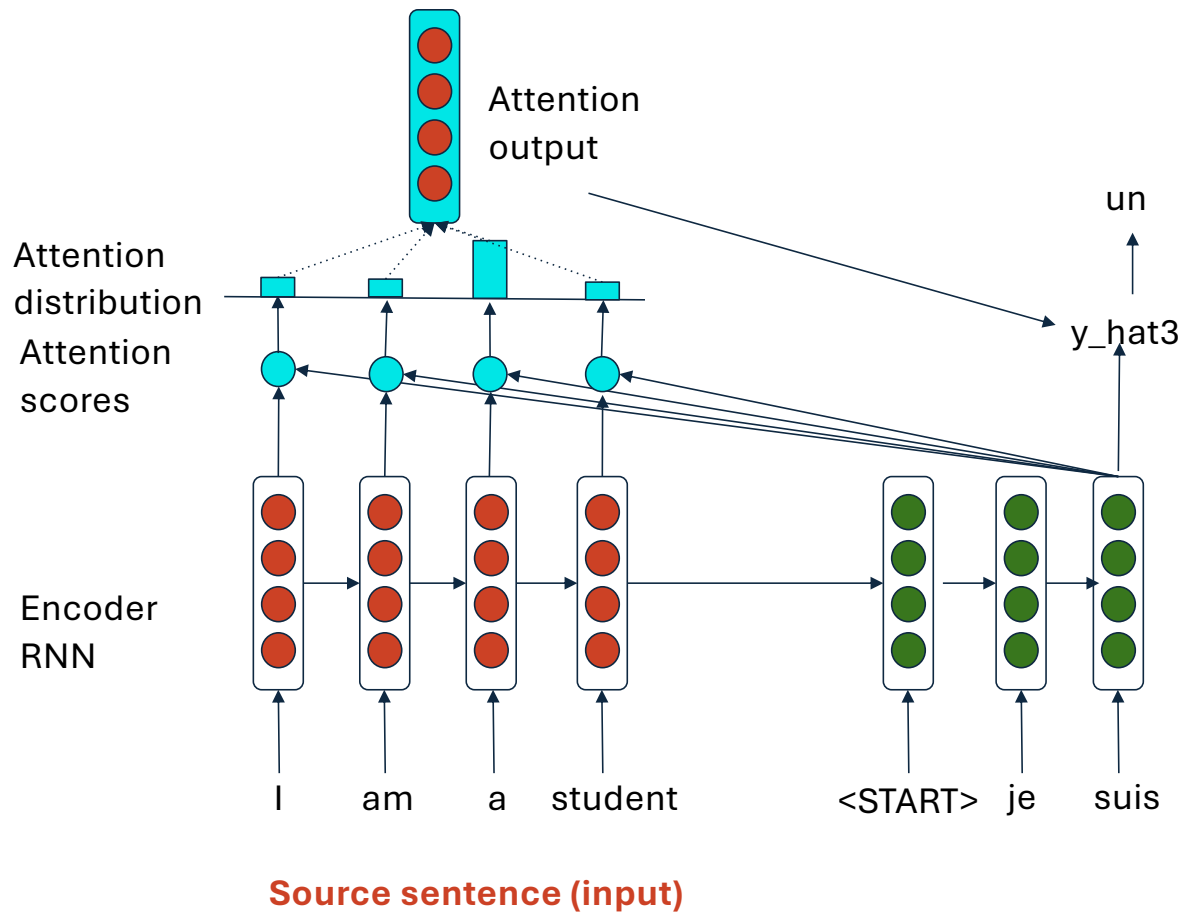
Seq2Seq with attention



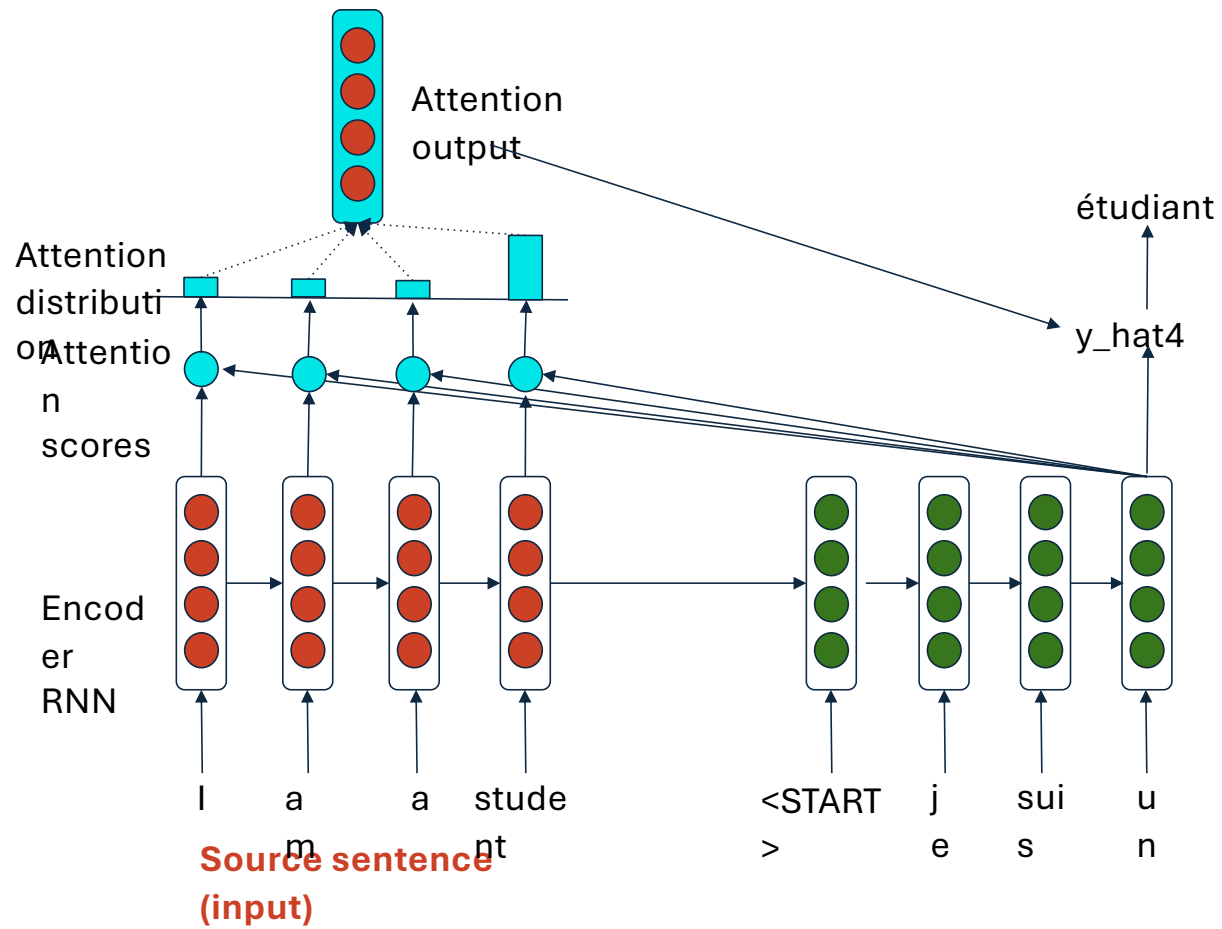
Seq2Seq with attention



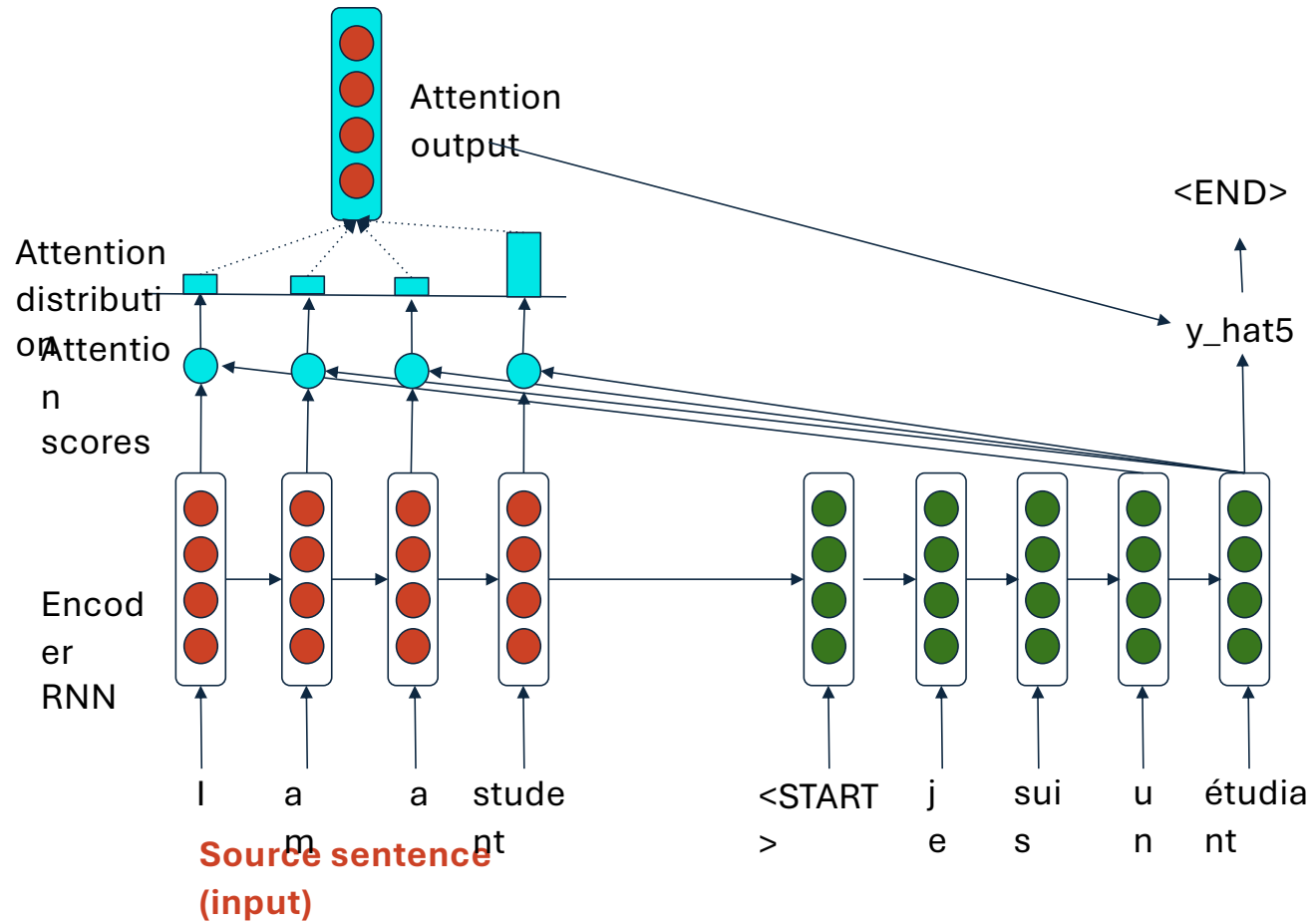
Seq2Seq with attention



Seq2Seq with attention



Seq2Seq with attention



Attention: in equations

- We have encoder hidden states $h_1, \dots, h_N \in \mathbb{R}^h$
- On timestep t , we have decoder hidden state $s_t \in \mathbb{R}^h$
- We get the **attention scores** e^t for this step:

$$e^t = [s_t^T h_1, \dots, s_t^T h_N] \in \mathbb{R}^N$$

- We take softmax to get the **attention distribution** α^t for this step (this is a probability distribution and sums to 1)

$$\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^N$$

Attention: in equations

- We use α^t to take a weighted sum of the encoder hidden states to get the **attention output** $\mathbf{a}_t$

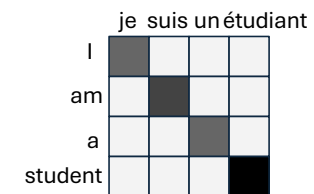
$$\mathbf{a}_t = \sum_{i=1}^N \alpha_i^t \mathbf{h}_i \in \mathbb{R}^h$$

- Finally, we concatenate the attention output $\mathbf{a}_t$ with the decoder hidden state $\mathbf{s}_t$ and proceed as in the non-attention seq2seq model

$$\underline{[\mathbf{a}_t; \mathbf{s}_t]} \in \mathbb{R}^{2h}$$

Attention is great

- Attention significantly **improves NMT performance**
 - Decoder focuses on certain parts of the source
- Attention solves the **bottleneck problem**
 - Decoder looks directly at source; bypass bottleneck
- Attention helps with **vanishing gradient problem**
 - Provides shortcut to far away states
- Attention provides some **interpretability**
 - Inspecting attention distribution
 - We get (soft) alignment for free!
 - No need to explicitly train an alignment system



Attention is a general Deep Learning technique

- We've seen that attention is a great way to improve the sequence-to-sequence model for Machine Translation.
- We can use attention in many **architectures** (not just seq2seq) and many **tasks** (not just MT)

Attention is a general Deep Learning technique

- More general definition of attention:

Given a set of vector *values*, and a vector *query*, attention is a technique to compute a weighted sum of the values, dependent on the query.

- We sometimes say that the query attends to the values.
 - For example, in the seq2seq + attention model, each decoder hidden state (*query*) attends to all the encoder hidden states (*values*).

Attention is a general Deep Learning technique

- More general definition of attention:

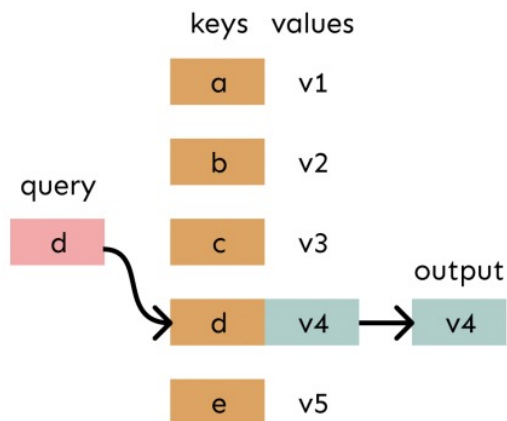
Given a set of vector *values*, and a vector *query*, attention is a technique to compute a weighted sum of the values, dependent on the query.

- Intuition:
 - A selective summary of the information contained in the values, where the query determines which values to focus on.
 - A fixed-size representation of an arbitrary set of representations (the values), **dependent on** some other representation (the query).

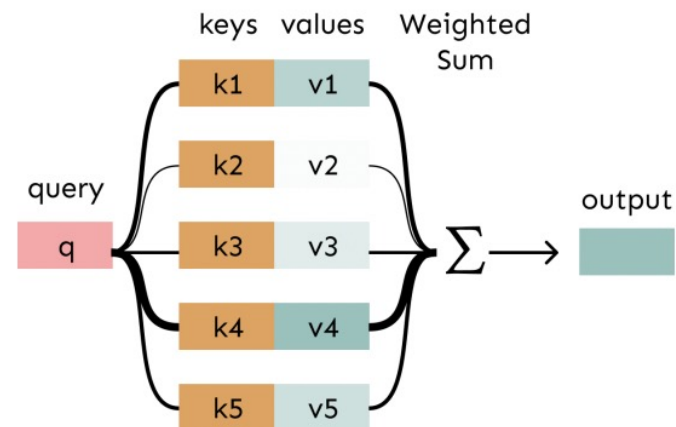
Attention as a soft, averaging lookup table

We can think of attention as performing **fuzzy lookup** in a key-value store.

In a **lookup table**, we have a table of **keys** that map to **values**. The **query** matches one of the keys, returning its value.



In **attention**, the **query** matches all **keys** *softly*, to a weight between 0 and 1. The keys' **values** are multiplied by the weights and summed.



Source: <https://web.stanford.edu/class/cs224n/slides/cs224n-2023-lecture08-transformers.pdf>

Attention variants

There are several ways you can compute $e \in \mathbb{R}^N$ from $h_1, \dots, h_N \in \mathbb{R}^{d_1}$ and $s \in \mathbb{R}^{d_2}$:

- Basic dot-product attention: $e_i = s^T h_i \in \mathbb{R}$
 - Note: this assumes $d_1 = d_2$
 - This is the version we saw earlier
- Multiplicative attention: $e_i = s^T W h_i \in \mathbb{R}$
 - Where $W \in \mathbb{R}^{d_2 \times d_1}$ is a weight matrix
- Additive attention: $e_i = v^T \tanh(W_1 h_i + W_2 s) \in \mathbb{R}$
 - Where $W_1 \in \mathbb{R}^{d_3 \times d_1}$, $W_2 \in \mathbb{R}^{d_3 \times d_2}$ are weight matrices and $v \in \mathbb{R}^{d_3}$ is a weight vector.

Attention variants

- We have some values $\mathbf{h}_1, \dots, \mathbf{h}_N \in \mathbb{R}^{d_1}$ and a query $\mathbf{s} \in \mathbb{R}^{d_2}$
- Attention always involves:
 1. Computing the **attention scores** $\mathbf{e} \in \mathbb{R}^N$
 2. Taking softmax to get attention distribution α :

There are multiple ways to do this

$$\alpha = \text{softmax}(\mathbf{e}) \in \mathbb{R}^N$$

3. Using attention distribution to take weighted sum of values:

$$\mathbf{a} = \sum_{i=1}^N \alpha_i \mathbf{h}_i \in \mathbb{R}^{d_1}$$

thus obtaining the attention output $\mathbf{a}$ (sometimes called the **context vector**)

Readings

- **CH 10 DL; CH 17 NNLP**
- Sutskever et al. [Sequence to Sequence Learning with Neural Networks](#). 2014 (original seq2seq NMT paper)
- Papineni et al. [BLEU: a Method for Automatic Evaluation of Machine Translation](#). 2002.
- Alex Graves. [Sequence Transduction with Recurrent Neural Networks](#) (early seq2seq speech recognition paper)